

Performance Analysis

 Reattempt

 Download Certificate

Detailed analysis of your performance in the AWS Certified Solutions Architect Associate - SAA C03 exam.

 Answer Distribution

0
Correct

0
Incorrect

65
Unattempted

 Domain Performance

Networking	0%
Security	0%
Databases	0%
Cost Optimization	0%
Storage	0%
Compute	0%
Monitoring and Observability	0%
Serverless	0%

Final Score

**0/65**

Passing score: 48

Time Taken

**130m 0s**

Total time: 130 minutes

Question Analysis



ITcertifications



Incorrect

0

Unattempted

65

Reviewed

0

Question-wise Analysis

Expand All

Collapse All

Question 1

Not Attempted

Networking



A company is migrating its data and applications to AWS over several years. They require secure access to data on Amazon S3 from both their AWS Region and their on-premises environment, without traversing the internet. An AWS Direct Connect connection is established between their AWS Region and on-premises location. Which solution satisfies these requirements?

- ☐ Create gateway endpoints for Amazon S3. Use the gateway endpoints to securely access the data from the Region and the on-premises location.
Gateway endpoints are for accessing S3 from within a VPC and do not extend connectivity to on-premises locations.
- ☐ Create a gateway in AWS Transit Gateway to access Amazon S3 securely from the Region and the on-premises location.
Transit Gateway is for routing traffic between VPCs and on-premises networks, not for direct S3 access.
- ☒ Create interface endpoints for Amazon S3. Use the interface endpoints to securely access the data from the Region and the on-premises location.
Interface endpoints provide private connectivity to S3, accessible from both the AWS Region and on-premises via Direct Connect, without using the internet.
- ☐ Use an AWS Key Management Service (AWS KMS) key to access the data securely from the Region and the on-premises location.
KMS is for managing encryption keys, not for providing network access to S3.

Overall Explanation:

Interface endpoints (AWS PrivateLink) allow private connectivity from your VPC and on-premises locations (via Direct Connect) to Amazon S3 without exposing your traffic to the public internet. Gateway endpoints are limited to VPC-internal access. Transit Gateway and KMS do not directly address the requirement of secure, private S3 access from on-premises.

[View Discussions](#)[Ask AI](#)

Question 2

Not Attempted

Security



A company has created a new organization in AWS Organizations with multiple accounts for development teams. These teams use AWS IAM Identity Center (AWS Single Sign-On) to access the accounts. For each application, the development teams must tag created resources with a predefined application name. A solutions architect needs to design a solution that allows the development teams to create resources only if the application name tag has an approved value. Which solution meets these requirements?

- ⊗ Create an IAM group that has a conditional Allow policy that requires the application name tag to be specified for resources to be created.

While an IAM policy can enforce the presence of a tag, it cannot restrict the values of the tag to a predefined list. This approach does not fully meet the requirement.

- ⊗ Create a cross-account role that has a Deny policy for any resource that has the application name tag.

This option would prevent the use of the tag entirely, which is contrary to the requirement to enforce specific approved values.

- ⊗ Create a resource group in AWS Resource Groups to validate that the tags are applied to all resources in all accounts.

AWS Resource Groups can help organize and manage resources based on tags, but it doesn't prevent the creation of resources with invalid tag values.

- ✔ Create a tag policy in Organizations that has a list of allowed application names.

Tag policies in AWS Organizations allow you to define rules for tags that are applied to resources.

You can specify allowed values for a tag key and enforce compliance, preventing resource creation if the tag doesn't conform to the policy.

Overall Explanation:

Tag policies in AWS Organizations are the most suitable solution because they allow you to define allowed values for tags and enforce compliance across multiple accounts. This ensures that development teams can only create resources with approved application name tag values.

https://docs.aws.amazon.com/organizations/latest/userguide/orgs_manage_policies_tag-policies.html

 View Discussions

 Ask AI

Question 3

Not Attempted

Databases



A company runs its databases on Amazon RDS for PostgreSQL. The company needs a secure and automated solution to manage the master user password by rotating it every 30 days with the least operational overhead. Which solution will meet these requirements?

- ⊗ Use Amazon EventBridge to schedule a custom AWS Lambda function to rotate the password every 30 days.

Using Lambda introduces more operational overhead for managing the function, its permissions, and ensuring its reliability.

- ⊗ Use the `modify-db-instance` command in the AWS CLI to change the password.

Using the AWS CLI requires manual intervention or custom scripting, leading to increased operational overhead and potential security risks if not handled properly.

- ✔ Integrate AWS Secrets Manager with Amazon RDS for PostgreSQL to automate password rotation.

AWS Secrets Manager is designed for this purpose and provides automated password rotation with minimal operational overhead, making it the ideal solution.

- ⊗ Integrate AWS Systems Manager Parameter Store with Amazon RDS for PostgreSQL to automate password rotation.

While Parameter Store can store secrets, it doesn't offer built-in password rotation capabilities like Secrets Manager, requiring more configuration and management.

Overall Explanation:

AWS Secrets Manager provides built-in integration with Amazon RDS for PostgreSQL for automated password rotation. This solution has the least operational overhead because it is a managed service specifically designed for this purpose. EventBridge and Lambda requires more overhead, CLI requires manual effort, and Parameter Store is less specialized for password rotation than Secrets Manager.

 View Discussions

 Ask AI

Question 4

Not Attempted

Databases



A company hosts an application on multiple Amazon EC2 instances. The application processes messages from an Amazon SQS queue, writes to an Amazon RDS table, and deletes the message from the queue. Occasional duplicate records are found in the RDS table. The SQS queue does not contain any duplicate messages. What should a solutions architect do to ensure messages are being processed once only?

- ☐ Use the CreateQueue API call to create a new queue.
Creating a new queue does not address the issue of duplicate message processing within the existing architecture.
- ☐ Use the AddPermission API call to add appropriate permissions.
Adding permissions is not relevant to preventing duplicate message processing. Permissions control access to the queue, not the processing of messages.
- ☐ Use the ReceiveMessage API call to set an appropriate wait time.
Setting an appropriate wait time can help manage the rate at which messages are received, but it does not prevent duplicate processing if the visibility timeout is too short.
- ☒ Use the ChangeMessageVisibility API call to increase the visibility timeout.
Increasing the visibility timeout gives the consumer more time to process the message before it becomes visible to other consumers, reducing the chance of duplicate processing.

Overall Explanation:

The visibility timeout is the duration during which SQS prevents other consumers from receiving and processing the same message. By increasing the visibility timeout using the ChangeMessageVisibility API call, you allow more time for the processing of a message to complete before it becomes visible to other consumers. This reduces the chances of duplicate processing and the creation of duplicate records in the RDS table.

[View Discussions](#)[Ask AI](#)

Question 5

Not Attempted

Cost Optimization



A company performs tests on an application that uses an Amazon DynamoDB table. The tests run for 4 hours once a week. The company knows how many read and write operations the application performs to the table each second during the tests. The company does not currently use DynamoDB for any other use case. A solutions architect needs to optimize the costs for the table. Which solution will meet these requirements?

- ☐ Choose on-demand mode. Update the read and write capacity units appropriately.
On-demand mode is suitable for unpredictable workloads. It also doesn't allow you to update the read and write capacity units manually.

- ☒ Choose provisioned mode. Update the read and write capacity units appropriately.
Provisioned mode allows specifying read and write capacity. Since the read and write operations are known, provisioned mode offers cost optimization and updating read/write capacity will further help to reduce costs.
- ☐ Purchase DynamoDB reserved capacity for a 1-year term.
Purchasing reserved capacity is generally for consistent, high-utilization scenarios. It is not cost-effective for only 4 hours of usage per week.
- ☐ Purchase DynamoDB reserved capacity for a 3-year term.
Purchasing reserved capacity is generally for consistent, high-utilization scenarios. It is not cost-effective for only 4 hours of usage per week.

Overall Explanation:

Provisioned mode is best suited when the read/write capacity is predictable and known. Updating the read and write capacity units will optimize the costs for the table. On-demand is not a good fit for predictable workloads as DynamoDB On-Demand pricing is more expensive than provisioned capacity. Reserved capacity is not cost-effective, as the table will remain idle for most of the time.

[View Discussions](#)[Ask AI](#)

Question 6

Not Attempted

Cost Optimization



A company runs its applications on Amazon EC2 instances. The company performs periodic financial assessments of its AWS costs and recently identified unusual spending. The company needs a solution to prevent unusual spending in the future. The solution must monitor costs and notify responsible stakeholders in the event of unusual spending. Which solution will meet these requirements?

- ☐ Use an AWS Budgets template to create a zero spend budget.
Creating a zero-spend budget won't allow any spending and isn't practical for running applications. It also doesn't inherently provide anomaly detection and notifications.
- ☒ Create an AWS Cost Anomaly Detection monitor in the AWS Billing and Cost Management console.

AWS Cost Anomaly Detection automatically detects unusual spending patterns based on machine learning. It identifies anomalies and sends notifications, meeting the requirements.

- ⊗ Create AWS Pricing Calculator estimates for the current running workload pricing details.

AWS Pricing Calculator is used for estimating costs, not for continuous monitoring and anomaly detection of actual spending.

- ⊗ Use Amazon CloudWatch to monitor costs and to identify unusual spending.

While CloudWatch can monitor metrics related to AWS services, it doesn't inherently provide cost anomaly detection. Setting up anomaly detection with CloudWatch would require custom configurations and thresholds.

Overall Explanation:

AWS Cost Anomaly Detection is the most suitable solution. It's specifically designed to detect unusual spending patterns and notify stakeholders automatically. The other options don't directly address the need for anomaly detection and notifications in the same way.

 View Discussions

 Ask AI

Question 7

Not Attempted

Storage



A marketing company receives a large amount of new clickstream data in Amazon S3 from a marketing campaign. The company needs to analyze the clickstream data in Amazon S3 quickly to determine whether to process the data further in the data pipeline. Which solution will meet these requirements with the LEAST operational overhead?

- ⊗ Create external tables in a Spark catalog. Configure jobs in AWS Glue to query the data.
This option involves managing Spark jobs and a Spark catalog, which increases operational overhead.

- ✔ Configure an AWS Glue crawler to crawl the data. Configure Amazon Athena to query the data.

This option utilizes AWS Glue for schema discovery and Amazon Athena for serverless querying directly from S3, minimizing operational overhead.

- ⊗ Create external tables in a Hive metastore. Configure Spark jobs in Amazon EMR to query the data.

This option requires managing an EMR cluster and a Hive metastore, resulting in high operational overhead.

- ⊗ Configure an AWS Glue crawler to crawl the data. Configure Amazon Kinesis Data Analytics to use SQL to query the data.

Kinesis Data Analytics is designed for analyzing streaming data. Since the data is already at rest in S3, using Kinesis Data Analytics is not efficient and adds unnecessary complexity.

Overall Explanation:

The optimal solution is to use AWS Glue to crawl the S3 data and create a schema, then use Amazon Athena to directly query the data in S3 using SQL. This approach leverages serverless services, minimizing management tasks and operational overhead compared to managing Spark clusters or using Kinesis Data Analytics for data at rest.

 View Discussions

 Ask AI

Question 8

Not Attempted

Storage



A company hosts an SMB file server in its data center. The file server stores frequently accessed large files for 7 days after creation. After 7 days, the files still need to be accessible, but with a maximum retrieval time of 24 hours. Which AWS solution meets these requirements?

- ⊗ Use AWS DataSync to copy data older than 7 days from the SMB file server to AWS.

AWS DataSync is used for data transfer and synchronization between on-premises storage and AWS, not for archival and retrieval according to specified timeframes.

- ✔ Create an Amazon S3 File Gateway to increase the company's storage space. Create an S3 Lifecycle policy to transition the data to S3 Glacier Deep Archive after 7 days.

Amazon S3 File Gateway supports SMB connections, allowing access to data stored in S3. An S3 Lifecycle policy can automatically transition older data to S3 Glacier Deep Archive, which offers cost-effective archival storage with retrieval times within 24 hours.

- ⊗ Create an Amazon FSx File Gateway to increase the company's storage space. Create an Amazon S3 Lifecycle policy to transition the data after 7 days.

Amazon FSx File Gateway does not store data directly in S3. It's designed to work with FSx for Windows File Server, not S3. Therefore, this option is incorrect.

- ⊗ Configure access to Amazon S3 for each user. Create an S3 Lifecycle policy to transition the data to S3 Glacier Flexible Retrieval after 7 days.

While S3 Glacier Flexible Retrieval can be used for archival, this option doesn't explain how the initial data transfer from the SMB file server to S3 would happen, nor does it offer a seamless SMB interface.

Overall Explanation:

Amazon S3 File Gateway provides the required SMB interface to access data. S3 Lifecycle policies allow for automatic transitioning of data to S3 Glacier Deep Archive, which meets the requirement of access within 24 hours for older files. Therefore, option B is the correct solution.

 View Discussions

 Ask AI

Question 9

Not Attempted

Databases



A company hosts a web application on Amazon EC2 instances within an Auto Scaling group. The application relies on an Amazon RDS for PostgreSQL database. During peak traffic, the application's performance degrades due to a significant read load on the database. Which two actions should a solutions architect implement to improve performance?

- ⊗ Turn on auto scaling for the DB instance.
RDS DB instance Autoscaling is not designed to address heavy read workload issues. It primarily helps with capacity issues related to storage or compute, not read performance.
- ✓ Create a read replica for the DB instance. Configure the application to send read traffic to the read replica.
Creating a read replica offloads read traffic from the primary DB instance, distributing the load and improving overall performance during periods of heavy read traffic.
- ⊗ Convert the DB instance to a Multi-AZ DB instance deployment. Configure the application to send read traffic to the standby DB instance.
You cannot directly send read traffic to the standby DB instance in a Multi-AZ deployment. The standby instance is for failover purposes.
- ✓ Create an Amazon ElastiCache cluster. Configure the application to cache query results in the ElastiCache cluster.

Amazon ElastiCache can be used to cache frequently accessed data, reducing the load on the database by serving read requests from the cache.

- ⊗ Configure the Auto Scaling group subnets to ensure that the EC2 instances are provisioned in the same Availability Zone as the DB instance.

Provisioning EC2 instances in the same AZ as the DB instance might reduce latency, but it doesn't address the root cause of the performance issue, which is the heavy read load on the database.

Overall Explanation:

To resolve the performance issues caused by heavy read loads on the RDS for PostgreSQL database, the solutions architect should implement the following actions: 1. **Create a read replica:** This allows the application to direct read traffic to the read replica, reducing the load on the primary database instance. 2. **Create an Amazon ElastiCache cluster:** Caching frequently accessed data in ElastiCache will further reduce the load on the database by serving read requests from the cache instead of querying the database directly.

 View Discussions

 Ask AI

Question 10

Not Attempted

Storage



A company uses Amazon EC2 instances and Amazon Elastic Block Store (Amazon EBS) volumes to run an application. The company creates one snapshot of each EBS volume every day to meet compliance requirements. The company wants to implement an architecture that prevents the accidental deletion of EBS volume snapshots. The solution must NOT change the administrative rights of the storage administrator user. Which solution will meet these requirements with the LEAST administrative effort?

- ⊗ Create an IAM role that has permission to delete snapshots. Attach the role to a new EC2 instance. Use the AWS CLI from the new EC2 instance to delete snapshots.
- This solution creates an overly complex process for deleting snapshots, and doesn't prevent accidental deletion.

- ⊗ Create an IAM policy that denies snapshot deletion. Attach the policy to the storage administrator user.

This solution changes the administrative rights of the storage administrator, which is against the requirements.

- ⊗ Add tags to the snapshots. Create retention rules in Recycle Bin for EBS snapshots that have the tags.

While Recycle Bin can help with accidental deletion, it requires configuration and might not directly prevent deletion with the least administrative effort.

- ✓ Lock the EBS snapshots to prevent deletion.

Locking EBS Snapshots directly prevents accidental deletion with minimal administrative effort and without changing the storage administrator's permissions.

Overall Explanation:

Locking EBS snapshots is the simplest and most direct way to prevent accidental deletion without impacting the administrator's permissions. AWS allows you to lock resources, including EBS snapshots, to prevent accidental or malicious deletion. This solution has the least administrative overhead compared to the other options. The other options are either too complex or change administrator's rights.

[View Discussions](#)[Ask AI](#)

Question 11

Not Attempted

Networking



A company's application uses Network Load Balancers, Auto Scaling groups, Amazon EC2 instances, and databases that are deployed in an Amazon VPC. The company wants to capture information about traffic to and from the network interfaces in near real time in its Amazon VPC. The company wants to send the information to Amazon OpenSearch Service for analysis. Which solution will meet these requirements?

- ⊗ Create a log group in Amazon CloudWatch Logs. Configure VPC Flow Logs to send the log data to the log group. Use Amazon Kinesis Data Streams to stream the logs from the log group to OpenSearch Service.

While Kinesis Data Streams can provide near real-time data, CloudWatch Logs and Kinesis Data Firehose integration is the recommended approach for sending logs to OpenSearch.

- ✓ Create a log group in Amazon CloudWatch Logs. Configure VPC Flow Logs to send the log data to the log group. Use Amazon Kinesis Data Firehose to stream the logs from the log group to OpenSearch Service.

This is the correct answer. VPC Flow Logs capture traffic information, CloudWatch Logs provides storage and management, and Kinesis Data Firehose efficiently streams the logs to Amazon OpenSearch Service for analysis.

- ✗ Create a trail in AWS CloudTrail. Configure VPC Flow Logs to send the log data to the trail. Use Amazon Kinesis Data Streams to stream the logs from the trail to OpenSearch Service.

CloudTrail is for auditing API calls, not for capturing network traffic data from VPC Flow Logs.

- ✗ Create a trail in AWS CloudTrail. Configure VPC Flow Logs to send the log data to the trail. Use Amazon Kinesis Data Firehose to stream the logs from the trail to OpenSearch Service.

CloudTrail is for auditing API calls, not for capturing network traffic data from VPC Flow Logs.

Overall Explanation:

The correct solution is to use VPC Flow Logs to capture the network traffic data and send it to a CloudWatch Logs log group. Then, use Kinesis Data Firehose to stream the logs from the log group to Amazon OpenSearch Service for analysis. CloudTrail is not used for capturing network traffic; it is used for auditing API calls. While Kinesis Data Streams could be used, Kinesis Data Firehose is the more appropriate and commonly used service for streaming logs to OpenSearch.

 View Discussions

 Ask AI

Question 12

Not Attempted

Cost Optimization



A company is developing an application that runs on a production Amazon Elastic Kubernetes Service (Amazon EKS) cluster using On-Demand Instances within managed node groups. They require a dedicated EKS cluster for development, used infrequently for application resiliency testing. The development cluster must also have its nodes fully managed by EKS to simplify operations. Which solution is MOST cost-effective for meeting these requirements?

- ✓ Create a managed node group that contains only Spot Instances.

Using Spot Instances provides significant cost savings, ideal for infrequent testing. EKS fully manages the nodes in the node group.

- ⊗ Create two managed node groups. Provision one node group with On-Demand Instances. Provision the second node group with Spot Instances.

While providing a mix of instance types, this option is more expensive than using only Spot Instances, which aligns with the cost-effectiveness requirement for infrequent use.

- ⊗ Create an Auto Scaling group that has a launch configuration that uses Spot Instances. Configure the user data to add the nodes to the EKS cluster.

This solution involves self-managing the nodes, which contradicts the requirement for EKS to fully manage the nodes through managed node groups.

- ⊗ Create a managed node group that contains only On-Demand Instances.

Using only On-Demand Instances is the most expensive option and does not align with the cost-effectiveness requirement.

Overall Explanation:

The MOST cost-effective solution is to create a managed node group consisting solely of Spot Instances. Spot Instances provide the lowest cost option, suitable for infrequent resiliency testing. Managed node groups ensure that EKS handles node management, fulfilling all requirements. Options B and D are more expensive, while option C does not use EKS managed node groups.

[View Discussions](#)[Ask AI](#)

Question 13

Not Attempted

Security



A company stores sensitive data in Amazon S3. A solutions architect needs to create an encryption solution. The company needs to fully control the ability of users to create, rotate, and disable encryption keys with minimal effort for any data that must be encrypted. Which solution will meet these requirements?

- ⊗ Use default server-side encryption with Amazon S3 managed encryption keys (SSE-S3) to store the sensitive data.

SSE-S3 does not allow the company to fully control the ability of users to create, rotate, and disable encryption keys.

- ☒ Create a customer managed key by using AWS Key Management Service (AWS KMS). Use the new key to encrypt the S3 objects by using server-side encryption with AWS KMS keys (SSE-KMS).
This option provides full control over key lifecycle management, including the ability to create, rotate, and disable keys. SSE-KMS integrates with AWS IAM for fine-grained access control.
- ☐ Create an AWS managed key by using AWS Key Management Service (AWS KMS). Use the new key to encrypt the S3 objects by using server-side encryption with AWS KMS keys (SSE-KMS).
AWS managed keys do not allow the company to fully control the ability of users to create, rotate, and disable encryption keys.
- ☐ Download S3 objects to an Amazon EC2 instance. Encrypt the objects by using customer managed keys. Upload the encrypted objects back into Amazon S3.
This option requires significant effort and is not the most efficient way to encrypt data in S3.

Overall Explanation:

The correct answer is to use SSE-KMS with a Customer Managed Key. This allows for full control over key management (creation, rotation, and disabling) with minimal effort. Options A and C don't provide the required level of control, and option D is inefficient. SSE-KMS with a customer-managed key integrates with IAM for fine-grained access control, meeting the security requirements.

[View Discussions](#)[Ask AI](#)

Question 14

Not Attempted

[Storage](#)

A company wants to back up its on-premises virtual machines (VMs) to AWS. The company's backup solution exports on-premises backups to an Amazon S3 bucket as objects. The S3 backups must be retained for 30 days and must be automatically deleted after 30 days. Which combination of steps will meet these requirements?

- ☒ Create an S3 bucket that has S3 Object Lock enabled.
Enabling S3 Object Lock is necessary to prevent the deletion of objects before the 30-day retention period.

- ☐ Create an S3 bucket that has object versioning enabled.
Object versioning must be enabled to use S3 Object Lock and to ensure that objects can be managed through lifecycle policies for deletion.
- ☒ Configure a default retention period of 30 days for the objects.
Setting a retention period ensures that the objects are retained for the required 30 days using the object lock.
- ☐ Configure an S3 Lifecycle policy to protect the objects for 30 days.
An S3 lifecycle policy cannot directly protect objects. It is used for transitioning or expiring objects.
- ☒ Configure an S3 Lifecycle policy to expire the objects after 30 days.
Configuring an S3 Lifecycle policy to expire the objects after 30 days ensures that they are automatically deleted after the retention period.
- ☐ Configure the backup solution to tag the objects with a 30-day retention period
Tagging with a retention period will not automatically retain the objects, object lock is required.

Overall Explanation:

The correct combination of steps involves enabling S3 Object Lock to prevent early deletion (A), setting a 30-day retention period for those objects (C) and configuring a Lifecycle policy to automatically expire the objects after 30 days (E). Object Versioning is enabled automatically when Object Lock is enabled.

[View Discussions](#)[Ask AI](#)

Question 15

Not Attempted

Networking



A solutions architect is designing a new hybrid architecture to extend a company's on-premises infrastructure to AWS. The company requires a highly available connection with consistent low latency to an AWS Region. The company needs to minimize costs and is willing to accept slower traffic if the primary connection fails. What should the solutions architect do to meet these requirements?

- ☒ Provision an AWS Direct Connect connection to a Region. Provision a VPN connection as a backup if the primary Direct Connect connection fails.

This option provides a highly available, low-latency connection with Direct Connect, and a cost-effective backup using VPN. Direct Connect offers consistent performance, while the VPN provides redundancy at a lower cost when high performance isn't critical.

- ⊗ Provision a VPN tunnel connection to a Region for private connectivity. Provision a second VPN tunnel for private connectivity and as a backup if the primary VPN connection fails.

While VPN provides private connectivity, it doesn't offer the consistent low latency of Direct Connect. Two VPNs improve availability, but they don't meet the primary requirement for low latency.

- ⊗ Provision an AWS Direct Connect connection to a Region. Provision a second Direct Connect connection to the same Region as a backup if the primary Direct Connect connection fails.

This provides high availability and low latency, but it doesn't minimize costs, as two Direct Connect connections are expensive.

- ⊗ Provision an AWS Direct Connect connection to a Region. Use the Direct Connect failover attribute from the AWS CLI to automatically create a backup connection if the primary Direct Connect connection fails.

The Direct Connect failover attribute doesn't automatically create a backup connection. It configures how traffic is routed in case of a Direct Connect failure, but relies on existing connections.

Overall Explanation:

The best solution is to provision an AWS Direct Connect connection for high availability and low latency, and a VPN connection as a cost-effective backup when the primary connection fails. This meets all the requirements: high availability, consistent low latency during normal operation using Direct Connect, and minimized costs using a VPN for backup connectivity.

[View Discussions](#)[Ask AI](#)

Question 16

Not Attempted

Storage



A solutions architect needs to copy files from an Amazon S3 bucket to an Amazon Elastic File System (Amazon EFS) file system and another S3 bucket. The files must be copied continuously. New files are added to the original S3 bucket consistently. The copied files should be

overwritten only if the source file changes. Which solution will meet these requirements with the LEAST operational overhead?

- ☒ Create an AWS DataSync location for both the destination S3 bucket and the EFS file system. Create a task for the destination S3 bucket and the EFS file system. Set the transfer mode to transfer only data that has changed.
- AWS DataSync efficiently copies data between storage solutions. Setting the transfer mode to only transfer changed data minimizes data transfer and overhead.
- ☐ Create an AWS Lambda function. Mount the file system to the function. Set up an S3 event notification to invoke the function when files are created and changed in Amazon S3. Configure the function to copy files to the file system and the destination S3 bucket.
- While Lambda can achieve this, it involves more configuration and management overhead compared to DataSync. It requires managing the function, its scaling, and error handling.
- ☐ Create an AWS DataSync location for both the destination S3 bucket and the EFS file system. Create a task for the destination S3 bucket and the EFS file system. Set the transfer mode to transfer all data.
- Transferring all data on every run is inefficient and unnecessary, especially when only changed files need to be updated.
- ☐ Launch an Amazon EC2 instance in the same VPC as the file system. Mount the file system. Create a script to routinely synchronize all objects that changed in the origin S3 bucket to the destination S3 bucket and the mounted file system.
- Using an EC2 instance and custom script involves managing the instance, the script, and ensuring the synchronization process is reliable and efficient, leading to higher operational overhead.

Overall Explanation:

AWS DataSync offers the least operational overhead because it's a managed service designed for efficient data transfer. Setting the transfer mode to only copy changed data ensures that only new or modified files are copied, minimizing data transfer and management efforts compared to Lambda, EC2-based solutions, or transferring all data.

 View Discussions

 Ask AI

Question 17

Not Attempted

Security



A company uses Amazon EC2 instances and stores data on Amazon Elastic Block Store (Amazon EBS) volumes. The company must ensure that all data is encrypted at rest by using AWS Key Management Service (AWS KMS). The company must be able to control rotation of the encryption keys. Which solution will meet these requirements with the LEAST operational overhead?

- ☒ Create a customer managed key. Use the key to encrypt the EBS volumes.
Creating a customer managed key allows you to control key rotation and provides the necessary encryption at rest, while still being managed by you for key rotation policies.
- ☐ Use an AWS managed key to encrypt the EBS volumes. Use the key to configure automatic key rotation.
While AWS managed keys provide automatic key rotation, they do not offer the level of control that a customer managed key does. You cannot configure specific rotation policies with AWS managed keys.
- ☐ Create an external KMS key with imported key material. Use the key to encrypt the EBS volumes.
Importing key material adds significant operational overhead as you're responsible for generating, protecting, and importing the key material into KMS. This is more complex than simply creating a customer managed key within AWS KMS.
- ☐ Use an AWS owned key to encrypt the EBS volumes.
AWS owned keys do not allow for customer control over key rotation. The key rotation is entirely managed by AWS, which does not meet the requirement.

Overall Explanation:

The most suitable solution with the least operational overhead is to create a customer managed key. This option fulfills both the encryption at rest and key rotation control requirements. Options B and D do not allow company owned keys. Option C has too much operational overhead.

[View Discussions](#)[Ask AI](#)

Question 18

Not Attempted

Security



A company requires a solution to enforce data encryption at rest on Amazon EC2 instances. The solution must automatically identify noncompliant resources and enforce compliance policies on findings with the LEAST administrative overhead.

- ✔ Use an IAM policy that allows users to create only encrypted Amazon Elastic Block Store (Amazon EBS) volumes. Use AWS Config and AWS Systems Manager to automate the detection and remediation of unencrypted EBS volumes.

This approach proactively prevents the creation of unencrypted volumes using IAM. AWS Config detects non-compliant resources, and AWS Systems Manager Automation handles automated remediation, providing a comprehensive and streamlined solution with minimal overhead.

- ✘ Use AWS Key Management Service (AWS KMS) to manage access to encrypted Amazon Elastic Block Store (Amazon EBS) volumes. Use AWS Lambda and Amazon EventBridge to automate the detection and remediation of unencrypted EBS volumes.

While feasible, managing encryption access with KMS and automating detection/remediation with Lambda and EventBridge introduces more operational complexity and overhead than using AWS Config and Systems Manager.

- ✘ Use Amazon Macie to detect unencrypted Amazon Elastic Block Store (Amazon EBS) volumes. Use AWS Systems Manager Automation rules to automatically encrypt existing and new EBS volumes.

Amazon Macie is designed for identifying personally identifiable information (PII) and sensitive data, not for detecting unencrypted EBS volumes. Therefore, it's not the appropriate tool for this requirement.

- ✘ Use Amazon Inspector to detect unencrypted Amazon Elastic Block Store (Amazon EBS) volumes. Use AWS Systems Manager Automation rules to automatically encrypt existing and new EBS volumes.

Amazon Inspector is designed for identifying security vulnerabilities and deviations from security best practices, such as OS patching and vulnerability detection, not specifically for detecting unencrypted EBS volumes.

Overall Explanation:

The most efficient solution is to use an IAM policy to enforce encryption at creation and leverage AWS Config to identify non-compliant resources. AWS Systems Manager Automation then remediates findings. This approach provides proactive enforcement, automated detection, and streamlined remediation with the least administrative overhead. Macie is for PII detection, and Inspector is for vulnerability scanning, making them

unsuitable. Lambda/EventBridge adds unnecessary complexity compared to AWS Config's built-in capabilities.

[View Discussions](#)[Ask AI](#)

Question 19

Not Attempted

Databases



A company is migrating its multi-tier on-premises application to AWS. The application consists of a single-node MySQL database and a multi-node web tier. The company must minimize changes to the application during the migration. The company also wants to improve application resiliency after the migration. Which combination of steps will meet these requirements? (Choose two.)

- ☒ **Migrate the web tier to Amazon EC2 instances in an Auto Scaling group behind an Application Load Balancer.**
Migrating the web tier to Amazon EC2 instances in an Auto Scaling group behind an Application Load Balancer (ALB) provides horizontal scalability, automatic scaling, and improved resiliency. ALB is well-suited for routing traffic for web applications.
- ☐ **Migrate the database to Amazon EC2 instances in an Auto Scaling group behind a Network Load Balancer.**
While NLB can provide load balancing, it is not ideal for a MySQL database. NLB is best suited for TCP/UDP traffic and this approach does not directly address database resiliency needs in the simplest manner.
- ☒ **Migrate the database to an Amazon RDS Multi-AZ deployment.**
Migrating the database to an Amazon RDS Multi-AZ deployment provides high availability and automatic failover with minimal application changes. RDS Multi-AZ maintains a standby replica in a different Availability Zone, improving database resiliency.
- ☐ **Migrate the web tier to an AWS Lambda function.**
Migrating the web tier to AWS Lambda would likely require significant changes to the application architecture and code, which contradicts the requirement to minimize changes.
- ☐ **Migrate the database to an Amazon DynamoDB table.**
Migrating the database to Amazon DynamoDB would require significant changes to the application's data model and code, as DynamoDB is a NoSQL database. This contradicts the requirement to minimize changes.

Overall Explanation:

Migrating the web tier to EC2 instances in an Auto Scaling group behind an ALB (Option A) provides scalability and resilience for the web tier with minimal application changes.

Migrating the database to an RDS Multi-AZ deployment (Option C) enhances database availability and provides automatic failover with minimal changes to the application's database connectivity. Options D and E would require significant code and architecture changes. Option B is not optimal as the application consists of web tier and database and hence ALB is better than NLB for web application and also does not improve database resiliency compared to RDS Multi-AZ.

[View Discussions](#)[Ask AI](#)

Question 20

Not Attempted

Networking



A company headquartered near the eu-central-1 Region plans to migrate its web applications to AWS. Due to regulatory constraints, certain applications cannot be launched within eu-central-1. The company requires single-digit millisecond latency for these applications. Which solution satisfies these requirements?

- ☐ Deploy the applications in eu-central-1. Extend the company's VPC from eu-central-1 to an edge location in Amazon CloudFront.

This option is incorrect because the regulations prevent launching applications in eu-central-1. Also, extending the VPC to CloudFront edge locations is not possible and CloudFront is not intended for this use case.

- ☒ Deploy the applications in AWS Local Zones by extending the company's VPC from eu-central-1 to the chosen Local Zone.

This is the correct option. AWS Local Zones allow deploying applications closer to end-users, achieving single-digit millisecond latency, while residing outside the restricted eu-central-1 Region. The VPC can be extended to the Local Zone.

- ☐ Deploy the applications in eu-central-1. Extend the company's VPC from eu-central-1 to the regional edge caches in Amazon CloudFront.

This option is incorrect because the regulations prevent launching applications in eu-central-1. Also, extending the VPC to CloudFront regional edge caches is not possible and CloudFront is not intended for this use case.

- ⊗ Deploy the applications in AWS Wavelength Zones by extending the company's VPC from eu-central-1 to the chosen Wavelength Zone.

This option is incorrect. Wavelength Zones are designed for mobile applications with 5G networks, not general web applications, and extending the VPC is not the primary function.

Overall Explanation:

The best solution is to deploy the applications in AWS Local Zones by extending the company's VPC from eu-central-1 to the chosen Local Zone. AWS Local Zones place compute, storage, and database resources closer to end-users, enabling single-digit millisecond latency while adhering to regulatory restrictions preventing deployment in eu-central-1. Wavelength is suited for mobile 5G use cases, and directly deploying in eu-central-1 or using CloudFront edges is not viable due to the regulations or VPC limitations.

 View Discussions

 Ask AI

Question 21

Not Attempted

Databases



A company's ecommerce website experiences unpredictable traffic and relies on AWS Lambda functions to directly access a private Amazon RDS for PostgreSQL DB instance. The company needs to maintain consistent database performance and prevent Lambda invocations from overwhelming the database with excessive connections. What steps should a solutions architect take to fulfill these requirements?

- ⊗ Point the client driver at an RDS custom endpoint. Deploy the Lambda functions inside a VPC.

RDS custom endpoints are not designed to manage database connections for scaling Lambda functions.

- ✔ Point the client driver at an RDS Proxy endpoint. Deploy the Lambda functions inside a VPC.

RDS Proxy manages database connections, reducing the load on the RDS instance and enabling Lambda functions within a VPC to access the database.

- ⊗ Point the client driver at an RDS custom endpoint. Deploy the Lambda functions outside a VPC.

RDS custom endpoints are not designed to manage database connections for scaling Lambda functions, and Lambda functions outside a VPC cannot access an RDS instance inside a VPC.

- ⊗ Point the client driver at an RDS Proxy endpoint. Deploy the Lambda functions outside a VPC.

Lambda functions outside a VPC cannot access an RDS instance inside a VPC.

Overall Explanation:

RDS Proxy is designed to manage database connections, preventing overload from Lambda invocations. Deploying Lambda functions inside the VPC allows them to access the private RDS instance. A Lambda function that's outside of a VPC can't access an RDS instance that's inside a VPC. <https://repost.aws/knowledge-center/connect-lambda-to-an-rds-instance>

 View Discussions

 Ask AI

Question 22

Not Attempted

Networking



A company is developing an application and stores test data in multiple on-premises locations. The company needs to connect these on-premises locations to VPCs within an AWS Region. The number of accounts and VPCs is expected to increase significantly over the next year. The network architecture should simplify the administration of new connections and provide scalability. Which solution meets these requirements with the LEAST administrative overhead?

- ⊗ Create a peering connection between the VPCs. Create a VPN connection between the VPCs and the on-premises locations.

Peering connections become complex to manage with a large number of VPCs. Managing VPN connections individually for each VPC and on-premises location increases administrative overhead.

- ⊗ Launch an Amazon EC2 instance. On the instance, include VPN software that uses a VPN connection to connect all VPCs and on-premises locations.

Using an EC2 instance as a VPN concentrator introduces a single point of failure and requires significant manual configuration and management. This option does not scale well and adds administrative burden.

- ✓ Create a transit gateway. Create VPC attachments for the VPC connections. Create VPN attachments for the on-premises connections.

A transit gateway simplifies network management by acting as a central hub for connecting VPCs and on-premises locations. It reduces administrative overhead and provides scalability for an increasing number of accounts and VPCs.

- ⊗ Create an AWS Direct Connect connection between the on-premises locations and a central VPC. Connect the central VPC to other VPCs by using peering connections.

While Direct Connect provides dedicated connectivity, using a central VPC and peering connections doesn't scale efficiently and increases administrative overhead with a growing number of VPCs. Direct Connect alone doesn't solve the VPC to VPC connectivity issue.

Overall Explanation:

The transit gateway (Option C) is the most suitable solution because it simplifies the administration of new connections and provides the necessary scalability for an increasing number of accounts and VPCs. It acts as a central hub, making it easier to manage connectivity between on-premises locations and multiple VPCs. Options A and D involve more complex configurations with peering and individual VPN connections that are difficult to scale. Option B introduces a single point of failure and requires significant manual management. Therefore, transit gateway offers the least administrative overhead.

 View Discussions

 Ask AI

Question 23

Not Attempted

Compute



A company that uses AWS needs a solution to predict the resources needed for manufacturing processes each month. The solution must use historical values that are currently stored in an Amazon S3 bucket. The company has no machine learning (ML) experience and wants to use a managed service for the training and predictions. Which combination of steps will meet these requirements? (Choose two.)

- ⊗ Deploy an Amazon SageMaker model. Create a SageMaker endpoint for inference.

Deploying a SageMaker model requires ML experience, which the company lacks. While SageMaker is a managed service, deployment involves ML knowledge.

- ⊗ Use Amazon SageMaker to train a model by using the historical data in the S3 bucket.

Using SageMaker directly for training requires ML experience, contradicting the company's requirements. Although SageMaker is managed, model training involves ML knowledge.

- ⊗ Configure an AWS Lambda function with a function URL that uses Amazon SageMaker endpoints to create predictions based on the inputs.

This option requires the use of SageMaker endpoints, which implicitly means the company needs to create and train the model using SageMaker. Therefore it requires ML experience, contradicting the company's requirements.

- ✓ Configure an AWS Lambda function with a function URL that uses an Amazon Forecast predictor to create a prediction based on the inputs.

Amazon Forecast is a managed service that allows for generating predictions without needing prior ML experience. AWS Lambda can be used to integrate with Amazon Forecast.

- ✓ Train an Amazon Forecast predictor by using the historical data in the S3 bucket.

Amazon Forecast can be trained using data from S3. It is a fully managed service, so the company can train an Amazon Forecast predictor without needing ML experience.

Overall Explanation:

The best solution involves using Amazon Forecast because the company wants a managed service and has no ML experience. Option E, training an Amazon Forecast predictor using data in S3, is necessary to build the prediction model. Option D, configuring a Lambda function to use the trained Forecast predictor, is needed to make predictions. Therefore options D and E meet the requirements. Options A and B require ML experience, which the company lacks. Although Amazon Forecast is no longer available to new customers as of July 29, 2024, it may still appear as a valid service on the SAA-C03 exam.

 View Discussions

 Ask AI

Question 24

Not Attempted

Security



A company manages AWS accounts in AWS Organizations. AWS IAM Identity Center (AWS Single Sign-On) and AWS Control Tower are configured for the accounts. The company wants to manage multiple user permissions across all the accounts. The permissions will be used by multiple IAM users and must be split between the developer and administrator teams. Each team requires different permissions. The company wants a solution that includes new users that are hired on both teams. Which solution will meet these requirements with the LEAST operational overhead?

- ⊗ Create individual users in IAM Identity Center for each account. Create separate developer and administrator groups in IAM Identity Center. Assign the users to the appropriate groups. Create a custom IAM policy for each group to set fine-grained permissions.

This option requires managing users individually in each account, which increases operational overhead.

- ⊗ Create individual users in IAM Identity Center for each account. Create separate developer and administrator groups in IAM Identity Center. Assign the users to the appropriate groups. Attach AWS managed IAM policies to each user as needed for fine-grained permissions.

This option involves attaching managed policies to individual users in each account, leading to high operational overhead. Managed policies attached to users are not as scalable as Permission Sets.

- ✓ Create individual users in IAM Identity Center. Create new developer and administrator groups in IAM Identity Center. Create new permission sets that include the appropriate IAM policies for each group. Assign the new groups to the appropriate accounts. Assign the new permission sets to the new groups. When new users are hired, add them to the appropriate group.

This option centralizes user management and permission assignment through IAM Identity Center groups and permission sets, reducing operational overhead and enabling scalability for new users.

- ⊗ Create individual users in IAM Identity Center. Create new permission sets that include the appropriate IAM policies for each user. Assign the users to the appropriate accounts. Grant additional IAM permissions to the users from within specific accounts. When new users are hired, add them to IAM Identity Center and assign them to the accounts.

Creating permission sets for each user adds a high amount of overhead in a large environment and is not practical.

Overall Explanation:

Option C provides the least operational overhead by using IAM Identity Center to create users and groups, then defining permission sets with the appropriate IAM policies for each group. These permission sets are assigned to the relevant groups, and new users are simply added to the appropriate group to inherit the necessary permissions. This leverages the power of Identity Center to control account permissions centrally via Permission Sets.

Question 25

Not Attempted

Storage



A company wants to standardize its Amazon Elastic Block Store (Amazon EBS) volume encryption strategy. The company also wants to minimize the cost and configuration effort required to operate the volume encryption check. Which solution will meet these requirements?

- ☐ Write API calls to describe the EBS volumes and to confirm the EBS volumes are encrypted. Use Amazon EventBridge to schedule an AWS Lambda function to run the API calls.

This solution requires developing and maintaining custom code, and managing EventBridge schedules, which increases operational overhead and costs.
- ☐ Write API calls to describe the EBS volumes and to confirm the EBS volumes are encrypted. Run the API calls on an AWS Fargate task.

Similar to option A, this involves writing custom code and managing a Fargate task, adding operational complexity and potentially higher costs compared to a managed service.
- ☐ Create an AWS Identity and Access Management (IAM) policy that requires the use of tags on EBS volumes. Use AWS Cost Explorer to display resources that are not properly tagged. Encrypt the untagged resources manually.

IAM policies can enforce tagging, but Cost Explorer only helps identify untagged resources. This option does not directly enforce or automate encryption checking, and manual encryption is required.
- ☒ Create an AWS Config rule for Amazon EBS to evaluate if a volume is encrypted and to flag the volume if it is not encrypted.

AWS Config provides a managed service for assessing, auditing, and evaluating the configurations of AWS resources. It allows defining rules to automatically check the configuration of EBS volumes against desired configurations, such as encryption, ensuring consistent encryption with minimal manual effort.

Overall Explanation:

AWS Config allows defining rules to automatically check the configuration of AWS resources against desired configurations. By creating a custom AWS Config rule specifically for Amazon EBS volumes to evaluate if they are encrypted, you can ensure consistent encryption across all volumes. If a volume is found to be unencrypted, it can be flagged for further action. This solution automates the process of encryption checking, minimizing

manual effort and ensuring standardization across the environment. Additionally, AWS Config provides a cost-effective solution compared to continuously running scripts or tasks.

[View Discussions](#)[Ask AI](#)

Question 26

Not Attempted

Databases



A company hosts a business-critical web application on Amazon EC2 instances behind an Application Load Balancer. The EC2 instances reside in an Auto Scaling group. The application relies on an Amazon Aurora PostgreSQL database deployed in a single Availability Zone. To enhance resilience, the company aims to achieve high availability with minimal downtime and data loss in case of failures. Which solution will satisfy these requirements while minimizing operational overhead?

- ⊗ Place the EC2 instances in different AWS Regions. Use Amazon Route 53 health checks to redirect traffic. Use Aurora PostgreSQL Cross-Region Replication.

This approach, while providing geographic redundancy, introduces additional complexity in managing replication, handling latency, and potentially higher data transfer costs. Also, Aurora PostgreSQL DB clusters don't support Aurora Replicas in different AWS Regions.

- ✔ Configure the Auto Scaling group to use multiple Availability Zones. Configure the database as Multi-AZ. Configure an Amazon RDS Proxy instance for the database.

Configuring the Auto Scaling group across multiple Availability Zones ensures application availability even if one AZ fails. Configuring the database as Multi-AZ ensures automatic failover with minimal data loss. RDS Proxy improves connection management and scalability.

- ⊗ Configure the Auto Scaling group to use one Availability Zone. Generate hourly snapshots of the database. Recover the database from the snapshots in the event of a failure.

While snapshots offer data backup, they don't provide real-time failover and can lead to data loss between snapshots. Also restricting the EC2 instances to one AZ is against high availability.

- ⊗ Configure the Auto Scaling group to use multiple AWS Regions. Write the data from the application to Amazon S3. Use S3 Event Notifications to launch an AWS Lambda function to write the data to the database.

Distributing EC2 instances across regions and using S3 and Lambda adds complexity to the data flow and introduces additional overhead. Also EC2 Auto Scaling groups are regional constructs.

They can span Availability Zones, but not AWS regions.

Overall Explanation:

The optimal solution is to configure the Auto Scaling group across multiple Availability Zones and utilize Multi-AZ for the Aurora PostgreSQL database, along with an RDS Proxy. This combination ensures high availability, minimal downtime, and minimal data loss with the least operational effort. RDS Proxy is fully managed by AWS for RDS/Aurora. It is auto-scaling and highly available by default. Multi-AZ provides high scalability.

[View Discussions](#)[Ask AI](#)

Question 27

Not Attempted

Networking



A company uploads GB-sized files to Amazon S3 and then uses a fleet of Amazon EC2 Spot Instances to transcode the file format. The company needs to increase throughput during uploads from the on-premises data center to Amazon S3 and during downloads from Amazon S3 to the EC2 instances. Which solutions will meet these requirements? (Choose two.)

- ☐ Use S3 bucket access points instead of accessing the S3 bucket directly.
S3 Access Points simplify data access for services and applications but are not the best choice for increasing throughput in this scenario.
- ☐ Upload the files into multiple S3 buckets.
Uploading files into multiple S3 buckets can potentially increase throughput by distributing the load, but this is not an ideal solution for the requirements. Other options are better choices.
- ☒ Use S3 multipart uploads.
S3 multipart uploads allow you to upload large files in parts, improving throughput and resilience to network issues during the upload process.
- ☒ Fetch multiple byte-ranges of an object in parallel.
Fetching multiple byte-ranges of an object in parallel allows you to download large files in parts using concurrent connections, increasing download throughput.
- ☐ Add a random prefix to each object when uploading the files.
Adding random prefixes can improve performance by avoiding hot partitions, but it's not directly related to increasing upload or download throughput in the way the selected answers do.

Overall Explanation:

To meet the requirements of scaling throughput for both uploads to S3 and downloads to EC2 instances: * **S3 multipart uploads** allow the company to upload large files in smaller parts concurrently, which increases upload throughput. * **Fetching multiple byte-ranges of an object in parallel** allows the company to download the objects in smaller chunks concurrently, improving download throughput.

[View Discussions](#)[Ask AI](#)

Question 28

Not Attempted

Storage



A solutions architect is designing a shared storage solution for a web application that is deployed across multiple Availability Zones. The web application runs on Amazon EC2 instances within an Auto Scaling group. The application requires frequent content updates, and the solution must ensure strong consistency, delivering the new content immediately after changes are made. Which two solutions meet these requirements?

- ☐ Use AWS Storage Gateway Volume Gateway Internet Small Computer Systems Interface (iSCSI) block storage that is mounted to the individual EC2 instances.

Storage Gateway is typically used for hybrid environments connecting on-premises workloads to AWS. It is not optimized for a fully cloud-based solution and may introduce latency.

- ☒ Create an Amazon Elastic File System (Amazon EFS) file system. Mount the EFS file system on the individual EC2 instances.

Amazon EFS provides a fully managed, shared file storage system accessible across multiple Availability Zones, making it suitable for shared content and providing strong consistency.

- ☐ Create a shared Amazon Elastic Block Store (Amazon EBS) volume. Mount the EBS volume on the individual EC2 instances.

Sharing EBS volumes across multiple EC2 instances, even with Multi-Attach, can be complex to manage and doesn't provide the same level of availability and scalability as EFS.

- ☐ Use AWS DataSync to perform continuous synchronization of data between EC2 hosts in the Auto Scaling group.

DataSync is designed for periodic data synchronization, not for real-time updates or low-latency content delivery required for strong consistency.

- ✓ Create an Amazon S3 bucket to store the web content. Set the metadata for the Cache-Control header to no-cache. Use Amazon CloudFront to deliver the content.

Using S3 with CloudFront and a 'no-cache' Cache-Control header ensures that the latest content is always served. CloudFront provides fast, global delivery of the content. While S3 isn't a file system, for content delivery it can work. Setting no-cache ensures strong consistency.

Overall Explanation:

EFS offers a highly available, shared file system accessible across multiple AZs, providing a consistent view of the content for all EC2 instances. Using S3 with CloudFront and setting Cache-Control: no-cache guarantees immediate delivery of new content while leveraging CloudFront's edge locations for efficient distribution. Storage Gateway is better suited for hybrid environments, EBS multi-attach is complex, and DataSync is not designed for near real-time syncing.

 View Discussions

 Ask AI

Question 29

Not Attempted

Networking



A company is deploying an application in three AWS Regions using an Application Load Balancer (ALB). Amazon Route 53 will be used to distribute traffic between these Regions. What Route 53 routing policy should a solutions architect use to provide the *most high-performing* experience for users?

- ✓ Create an A record with a latency routing policy.

Latency-based routing directs traffic to the region with the lowest latency for the user, providing the best performance.

- ✗ Create an A record with a geolocation routing policy.

Geolocation routing directs traffic based on the geographic location of the user. While useful for localization, it doesn't necessarily guarantee the lowest latency.

- ✗ Create a CNAME record with a failover routing policy.

Failover routing is designed for disaster recovery scenarios, not for optimizing performance under normal operating conditions.

- ✗ Create a CNAME record with a geoproximity routing policy.

Geoproximity routing allows you to route traffic based on the geographic location of your users and resources and optionally specify bias, it might not result in the absolute lowest latency for each user.

Overall Explanation:

Latency-based routing (LBR) is the best choice for providing the most high-performing experience. It routes users to the AWS Region that offers the lowest latency, ensuring faster response times and improved user experience. Failover is for DR, and geolocation is mainly for local restrictions/content. Geoproximity is more complex and might not provide the lowest latency for all users. Using an 'A' record is suitable for ALBs as they resolve to IP addresses.

[View Discussions](#)[Ask AI](#)

Question 30

Not Attempted

[Databases](#)

A company's web application, hosted on EC2 instances behind an ALB, uses an embedded NoSQL database. These instances reside within an Auto Scaling group in a single Availability Zone. Increased traffic now demands high availability and eventual consistency for the database. Which solution best meets these requirements with the LEAST operational overhead?

- ☐ Replace the ALB with a Network Load Balancer. Maintain the embedded NoSQL database with its replication service on the EC2 instances.
Using a Network Load Balancer might improve performance, but maintaining the database replication across instances adds significant operational overhead, particularly for an embedded database.
- ☐ Replace the ALB with a Network Load Balancer. Migrate the embedded NoSQL database to Amazon DynamoDB by using AWS Database Migration Service (AWS DMS).
While migrating to DynamoDB is beneficial, using a Network Load Balancer is not the optimal choice for HTTP/HTTPS traffic. ALB is designed for this purpose.
- ☐ Modify the Auto Scaling group to use EC2 instances across three Availability Zones. Maintain the embedded NoSQL database with its replication service on the EC2 instances.
Distributing instances across multiple Availability Zones increases availability. However, maintaining replication for an embedded database across AZs is complex and operationally intensive.

- ✓ Modify the Auto Scaling group to use EC2 instances across three Availability Zones. Migrate the embedded NoSQL database to Amazon DynamoDB by using AWS Database Migration Service (AWS DMS).

Distributing instances across multiple AZs provides high availability. Migrating to DynamoDB offloads database management overhead and provides scalability and eventual consistency.

Overall Explanation:

The optimal solution combines distributing the application across multiple Availability Zones using the Auto Scaling group with migrating the NoSQL database to DynamoDB. This approach leverages DynamoDB's managed service benefits, including scalability, high availability, and eventual consistency, minimizing operational overhead. Using ALB is appropriate for the nature of web applications.

 View Discussions

 Ask AI

Question 31

Not Attempted

Databases



A company is building a shopping application on AWS. The application offers a catalog that changes once each month and needs to scale with traffic volume. The company wants the lowest possible latency from the application. Data from each user's shopping cart needs to be highly available. User session data must be available even if the user is disconnected and reconnects. What should a solutions architect do to ensure that the shopping cart data is preserved at all times?

- ✗ Configure an Application Load Balancer to enable the sticky sessions feature (session affinity) for access to the catalog in Amazon Aurora.
- Sticky sessions with an Application Load Balancer primarily ensure that requests from the same user are routed to the same backend instance. While it can help with session persistence, it does not guarantee the high availability of the shopping cart data, especially if the instance fails.
- ✓ Configure Amazon ElastiCache for Redis to cache catalog data from Amazon DynamoDB and shopping cart data from the user's session.
- Amazon ElastiCache for Redis is an in-memory data store that provides fast access to data and can be configured for high availability with replication. Storing the shopping cart data in ElastiCache for Redis ensures that the data is preserved even if the user disconnects and reconnects, and the high availability configuration prevents data loss in case of failures.

- ☐ **Configure Amazon OpenSearch Service to cache catalog data from Amazon DynamoDB and shopping cart data from the user's session.**

Amazon OpenSearch Service is primarily used for search and analytics, not for storing transactional session data. While it can store data, it is not designed for the low-latency access and high availability requirements of shopping cart data.

- ☐ **Configure an Amazon EC2 instance with Amazon Elastic Block Store (Amazon EBS) storage for the catalog and shopping cart. Configure automated snapshots.**

Storing shopping cart data on a single EC2 instance with EBS storage introduces a single point of failure. While snapshots can provide backup, they do not guarantee high availability or low latency during normal operation.

Overall Explanation:

Amazon ElastiCache for Redis provides a fast and highly available in-memory data store which will ensure that the shopping cart data is preserved at all times, even if a user disconnects and reconnects. The catalog data should also be cached in ElastiCache from DynamoDB as that is the database being used. ElastiCache can be configured to be highly available through replication, ensuring that the shopping cart data will not be lost. Therefore, configuring Amazon ElastiCache for Redis to cache catalog data from Amazon DynamoDB and shopping cart data from the user's session is the correct answer.

 [View Discussions](#)

 [Ask AI](#)

Question 32

Not Attempted

Monitoring and Observability



A company is building a microservices-based application that will be deployed on Amazon Elastic Kubernetes Service (Amazon EKS). The microservices will interact with each other. The company wants to ensure that the application is observable to identify performance issues in the future. Which solution will meet these requirements?

- ☐ **Configure the application to use Amazon ElastiCache to reduce the number of requests that are sent to the microservices.**

ElastiCache can reduce latency but does not directly provide observability.

- ☒ **Configure Amazon CloudWatch Container Insights to collect metrics from the EKS clusters. Configure AWS X-Ray to trace the requests between the microservices.**

CloudWatch Container Insights provides detailed metrics and logs for containers. AWS X-Ray traces requests across microservices, providing a complete observability solution.

- ⊗ Configure AWS CloudTrail to review the API calls. Build an Amazon QuickSight dashboard to observe the microservice interactions.

CloudTrail is designed for security auditing and API call tracking, not detailed observability or performance monitoring of microservice interactions.

- ⊗ Use AWS Trusted Advisor to understand the performance of the application.

Trusted Advisor focuses on high-level recommendations and does not provide real-time observability for application interactions or performance issues.

Overall Explanation:

CloudWatch Container Insights collects metrics, logs, and events from Amazon EKS clusters, providing monitoring and troubleshooting capabilities. AWS X-Ray collects data about requests that your application serves, and it helps you view, filter, and gain insights into that data to identify issues and opportunities for optimization. This combination provides the best observability for identifying performance issues in a microservices architecture.

[View Discussions](#)[Ask AI](#)

Question 33

Not Attempted

Security



A company needs to provide customers with secure access to data stored in an Amazon S3 bucket. The company processes customer data and the results must be encrypted at rest due to stringent regulations. Each customer should only access their own data from their AWS account, and company employees are prohibited from accessing the data. Which solution meets these requirements?

- ⊗ Provision an AWS Certificate Manager (ACM) certificate for each customer. Encrypt the data client-side. In the private certificate policy, deny access to the certificate for all principals except an IAM role that the customer provides.

ACM certificates are for securing communication, not for data encryption at rest. Client-side encryption and managing certificate policies are not the ideal solution for this scenario.

- ⊗ Provision a separate AWS Key Management Service (AWS KMS) key for each customer. Encrypt the data server-side. In the S3 bucket policy, deny decryption of data for all principals except an IAM role that the customer provides.

While KMS is appropriate for encryption, controlling decryption via S3 bucket policies is not the most effective or granular approach. KMS Key policies offer more precise control.

- ✔ Provision a separate AWS Key Management Service (AWS KMS) key for each customer. Encrypt the data server-side. In each KMS key policy, deny decryption of data for all principals except an IAM role that the customer provides.

Using KMS for server-side encryption with customer-specific keys provides data encryption at rest. Controlling decryption via KMS key policies ensures that only the specified customer IAM role can decrypt the data, preventing access from others, including company employees.

- ⊗ Provision an AWS Certificate Manager (ACM) certificate for each customer. Encrypt the data client-side. In the public certificate policy, deny access to the certificate for all principals except an IAM role that the customer provides.

ACM certificates are not for encrypting data at rest. Public certificate policies are generally not used for fine-grained access control over data encryption.

Overall Explanation:

The best solution is to provision a separate KMS key for each customer and encrypt the data server-side. The KMS key policy is configured to explicitly deny decryption to all principals except the IAM role that the specific customer provides. This ensures data encryption at rest, customer-specific access, and prevents unauthorized access, including company employees. ACM Certificates are for SSL/TLS, not encryption at rest.

 View Discussions

 Ask AI

Question 34

Not Attempted

Networking



A solutions architect creates a VPC with two public and two private subnets. Due to corporate security, EC2 instances must be launched in private subnets. An EC2 instance, hosting a web server on ports 80 and 443 within a private subnet, is unable to receive external internet traffic. What is the BEST solution to enable external access to the web server?

- ⊗ Attach the EC2 instance to an Auto Scaling group in a private subnet. Ensure that the DNS record for the website resolves to the Auto Scaling group identifier.

Auto Scaling groups manage instance scaling and availability, but do not directly address the issue of inbound traffic to a private subnet.

- ✔ Provision an internet-facing Application Load Balancer (ALB) in a public subnet. Add the EC2 instance to the target group that is associated with the ALB. Ensure that the DNS record for the website resolves to the ALB.

An internet-facing ALB in a public subnet can receive incoming internet traffic and forward it to the EC2 instance in the private subnet. The DNS record should resolve to the ALB, allowing external users to access the web server.

- ✘ Launch a NAT gateway in a private subnet. Update the route table for the private subnets to add a default route to the NAT gateway. Attach a public Elastic IP address to the NAT gateway.

A NAT gateway provides outbound internet access for instances in a private subnet, enabling them to initiate connections to the internet, such as for software updates. It does not enable inbound internet traffic.

- ✘ Ensure that the security group that is attached to the EC2 instance allows HTTP traffic on port 80 and HTTPS traffic on port 443. Ensure that the DNS record for the website resolves to the public IP address of the EC2 instance.

EC2 instances in a private subnet do not have public IP addresses, therefore directing the DNS record to the public IP address of the EC2 instance will not work. While security groups are important, they don't solve the fundamental problem of accessing a private instance from the internet.

Overall Explanation:

The BEST solution is to provision an internet-facing Application Load Balancer (ALB) in a public subnet. The ALB acts as an intermediary, receiving incoming traffic and routing it to the EC2 instance in the private subnet. This allows external users to access the web server while maintaining the security requirement of launching instances in private subnets. A NAT Gateway is used for outbound traffic from the private subnet. Autoscaling is irrelevant. Direct access via public IP to an EC2 instance in a private subnet is not possible.

[View Discussions](#)[Ask AI](#)

Question 35

Not Attempted

Storage



A company is deploying a new application to Amazon Elastic Kubernetes Service (Amazon EKS) with an AWS Fargate cluster. The application requires a storage solution for data persistence. The solution must be highly available and fault-tolerant, and it needs to be shared between multiple application containers within the Fargate pods. Which solution will meet these requirements with the LEAST operational overhead?

- ⊗ Create Amazon Elastic Block Store (Amazon EBS) volumes in the same Availability Zones where EKS worker nodes are placed. Register the volumes in a StorageClass object on an EKS cluster. Use EBS Multi-Attach to share the data between containers.

While EBS Multi-Attach allows sharing, EBS volumes are not inherently highly available or fault-tolerant across Availability Zones. This option also increases operational overhead due to the manual configuration and management required.

- ✔ Create an Amazon Elastic File System (Amazon EFS) file system. Register the file system in a StorageClass object on an EKS cluster. Use the same file system for all containers.

Amazon EFS provides a highly available, fault-tolerant, and shared storage solution with minimal operational overhead. It integrates well with EKS and can be easily configured for data persistence between multiple containers.

- ⊗ Create an Amazon Elastic Block Store (Amazon EBS) volume. Register the volume in a StorageClass object on an EKS cluster. Use the same volume for all containers.

EBS volumes can only be attached to a single EC2 instance at a time (without Multi-Attach, which has its own limitations), making it unsuitable for sharing data between multiple containers within different pods on a Fargate cluster.

- ⊗ Create Amazon Elastic File System (Amazon EFS) file systems in the same Availability Zones where EKS worker nodes are placed. Register the file systems in a StorageClass object on an EKS cluster. Create an AWS Lambda function to synchronize the data between file systems.

This approach introduces unnecessary complexity and operational overhead by requiring a Lambda function to synchronize data between multiple EFS file systems. EFS is inherently highly available, so synchronization is redundant and inefficient.

Overall Explanation:

Amazon EFS is the ideal choice because it inherently provides high availability, fault tolerance, and shared access, which directly addresses the requirements of the application. It also offers the least operational overhead, as it's a managed service that handles

replication and availability automatically. EBS, even with Multi-Attach, doesn't natively provide the HA/FT required across AZs and is harder to manage in this scenario.

[View Discussions](#)[Ask AI](#)

Question 36

Not Attempted

Storage



A company currently runs an application using Docker containers in its on-premises data center. The application's container instances rely on persistent data stored in a volume on the host machine. The company aims to migrate this application to a fully managed service to eliminate the need for managing servers or storage infrastructure. Which solution best fulfills these requirements?

- ☐ Use Amazon Elastic Kubernetes Service (Amazon EKS) with self-managed nodes. Create an Amazon Elastic Block Store (Amazon EBS) volume attached to an Amazon EC2 instance. Use the EBS volume as a persistent volume mounted in the containers.

This option involves managing EC2 instances and EBS volumes, which contradicts the requirement of a fully managed service and eliminating server/storage management.

- ☒ Use Amazon Elastic Container Service (Amazon ECS) with an AWS Fargate launch type. Create an Amazon Elastic File System (Amazon EFS) volume. Add the EFS volume as a persistent storage volume mounted in the containers.

This option uses Fargate, a serverless compute engine, and EFS, a fully managed file system. This aligns with the requirement of avoiding server and storage management. ECS with Fargate seamlessly integrates with EFS for persistent storage.

- ☐ Use Amazon Elastic Container Service (Amazon ECS) with an AWS Fargate launch type. Create an Amazon S3 bucket. Map the S3 bucket as a persistent storage volume mounted in the containers.

While Fargate is serverless, S3 is object storage and not designed to be directly mounted as a persistent volume for container workloads. Mounting S3 buckets directly to Fargate containers is not a standard or efficient practice and typically requires custom solutions.

- ⊗ Use Amazon Elastic Container Service (Amazon ECS) with an Amazon EC2 launch type. Create an Amazon Elastic File System (Amazon EFS) volume. Add the EFS volume as a persistent storage volume mounted in the containers.

Using the EC2 launch type means the company is responsible for managing the EC2 instances, which violates the requirement to have a fully managed service where the company does not manage servers.

Overall Explanation:

The correct answer is to use Amazon ECS with AWS Fargate and Amazon EFS. Fargate provides a serverless compute environment, meaning the company doesn't need to manage EC2 instances. EFS provides a fully managed network file system that can be used as a persistent volume for the containers. This combination avoids the need for managing servers or storage infrastructure. Mounting S3 buckets directly into Fargate containers is not a standard approach for persistent storage. Therefore, option B is the solution that best meets the requirements.

 View Discussions

 Ask AI

Question 37

Not Attempted

Serverless



A company's application ingests incoming messages consumed by dozens of microservices. The message volume fluctuates dramatically, spiking to 100,000 messages per second. To improve scalability and decouple the components, which solution is most suitable?

- ⊗ Persist the messages to Amazon Kinesis Data Analytics. Configure the consumer applications to read and process the messages.

Kinesis Data Analytics is designed for analyzing streaming data, not decoupling message ingestion and consumption. It's better suited for real-time analytics than handling high-volume message delivery to multiple consumers.

- ⊗ Deploy the ingestion application on Amazon EC2 instances in an Auto Scaling group. Scale the number of EC2 instances based on CPU metrics.

While Auto Scaling can improve scalability, it doesn't decouple the ingestion application from the consumer microservices. EC2 Auto Scaling also takes time to provision new instances, which might not be fast enough to handle sudden spikes to 100,000 messages per second.

- ✘ Write the messages to Amazon Kinesis Data Streams with a single shard. Use an AWS Lambda function to preprocess messages and store them in Amazon DynamoDB. Configure the consumer applications to read from DynamoDB.

Kinesis Data Streams can handle high volumes, but a single shard will be a bottleneck. Although Lambda can preprocess, the overall architecture doesn't decouple effectively, and DynamoDB introduces another potential bottleneck. Consumer applications reading directly from DynamoDB creates tight coupling.

- ✔ Publish the messages to an Amazon Simple Notification Service (Amazon SNS) topic with multiple Amazon Simple Queue Service (Amazon SQS) subscriptions. Configure the consumer applications to process the messages from the queues.

This solution uses SNS for publishing and SQS for queuing, providing decoupling and scalability. SNS distributes messages to SQS queues, allowing consumers to process messages independently at their own pace. The fan-out pattern inherent in SNS/SQS is ideal for decoupling publishers and subscribers, allowing the system to handle traffic spikes effectively.

Overall Explanation:

The most suitable solution is to use Amazon SNS and SQS. SNS provides a publish-subscribe mechanism, enabling decoupling of the ingestion application from the consumer applications. SQS then allows each consumer to process messages independently at their own pace. This design scales well for varying message volumes and decouples the services. Options A, B and C do not decouple the components or provide sufficient scalability compared to the SNS/SQS fan-out pattern. Although there may be questions of SNS API limits, this setup will be the most ideal.

 View Discussions

 Ask AI

Question 38

Not Attempted

Networking



A company's HTTP application is behind a Network Load Balancer (NLB). The NLB's target group is configured to use an Amazon EC2 Auto Scaling group with multiple EC2 instances that run the web service. The company notices that the NLB is not detecting HTTP errors for the application. These errors require a manual restart of the EC2 instances that run the web service. The company needs to improve the application's availability without writing custom scripts or code. What should a solutions architect do to meet these requirements?

- ⊗ Enable HTTP health checks on the NLB, supplying the URL of the company's application.

While NLBs support HTTP/HTTPS health checks, they are limited in functionality and don't interpret HTTP response codes like ALBs. This option might not effectively address the need for detecting HTTP errors.

- ⊗ Add a cron job to the EC2 instances to check the local application's logs once each minute. If HTTP errors are detected, the application will restart.

This approach involves custom scripting, which violates the requirement to avoid custom code. Additionally, it's a less elegant and potentially less reliable solution than using load balancer health checks.

- ✓ Replace the NLB with an Application Load Balancer. Enable HTTP health checks by supplying the URL of the company's application. Configure an Auto Scaling action to replace unhealthy instances.

This is the correct solution. ALBs support HTTP/HTTPS health checks that can detect HTTP errors based on response codes. Configuring Auto Scaling to replace unhealthy instances triggered by the ALB health checks ensures that only healthy instances serve traffic.

- ⊗ Create an Amazon CloudWatch alarm that monitors the UnhealthyHostCount metric for the NLB. Configure an Auto Scaling action to replace unhealthy instances when the alarm is in the ALARM state.

Since the NLB is not effectively detecting HTTP errors in the first place, monitoring the `UnhealthyHostCount` metric alone won't provide accurate information about the application's health. This approach is also less proactive than using ALB health checks.

Overall Explanation:

The best solution is to replace the NLB with an Application Load Balancer (ALB) and configure HTTP health checks. ALBs can interpret HTTP response codes, allowing them to detect application-level errors that an NLB cannot. Configuring an Auto Scaling action based on the ALB's health checks will ensure that unhealthy instances are automatically replaced, improving the application's availability without custom scripting.

 View Discussions

 Ask AI

Question 39

Not Attempted

Networking



A gaming company plans to deploy a new internet-facing application in multiple AWS Regions. The application relies on both TCP and UDP protocols for communication. The company's priorities are high availability and minimal latency for its global user base. Which combination of actions should a solutions architect take to meet these requirements?

- ☒ Create internal Network Load Balancers in front of the application in each Region.
Internal Network Load Balancers are suitable for traffic within the VPC, but don't directly address global routing and latency optimization.
- ☐ Create external Application Load Balancers in front of the application in each Region.
Application Load Balancers are suitable for HTTP/HTTPS traffic and do not support UDP. They also require additional services for global distribution.
- ☒ Create an AWS Global Accelerator accelerator to route traffic to the load balancers in each Region.
AWS Global Accelerator uses the AWS global network to direct traffic to the nearest healthy endpoint, minimizing latency and improving availability. It supports both TCP and UDP protocols.
- ☐ Configure Amazon Route 53 to use a geolocation routing policy to distribute the traffic.
Route 53 geolocation routing can distribute traffic based on user location, but it doesn't inherently optimize for latency or handle failover as effectively as Global Accelerator, especially for TCP and UDP.
- ☐ Configure Amazon CloudFront to handle the traffic and route requests to the application in each Region.
Amazon CloudFront is primarily designed for caching static and dynamic content, often using HTTP/HTTPS. It's not well-suited for routing generic TCP and UDP traffic for applications like gaming.

Overall Explanation:

The optimal solution involves using Network Load Balancers (NLBs) in each region to handle TCP/UDP traffic and AWS Global Accelerator to intelligently route traffic to the closest available NLB endpoint, thereby providing high availability and minimizing latency for global users. Using internal NLBs ensures traffic uses the AWS backbone network when routed to Global Accelerator. Global Accelerator is a good fit for non-HTTP use cases, such as gaming (UDP), IoT (MQTT), or Voice over IP.

[View Discussions](#)[Ask AI](#)

Question 40

Not Attempted

Security



A city has deployed a web application running on Amazon EC2 instances behind an Application Load Balancer (ALB). The application's users have reported sporadic performance, which appears to be related to DDoS attacks originating from random IP addresses. The city needs a solution that requires minimal configuration changes and provides an audit trail for the DDoS sources.

- ⊗ Enable an AWS WAF web ACL on the ALB, and configure rules to block traffic from unknown sources.

AWS WAF Web ACL contains WAF rules that define how to inspect web requests and what to do when a web request matches the inspection criteria. This option is not ideal because the DDoS attacks are originating from random IP addresses, making it difficult to define effective WAF rules.

- ⊗ Subscribe to Amazon Inspector. Engage the AWS DDoS Response Team (DRT) to integrate mitigating controls into the service.

Amazon Inspector is a service that analyzes your EC2 instances to identify potential security and configuration issues. It is not the most appropriate solution for mitigating DDoS attacks.

- ✔ Subscribe to AWS Shield Advanced. Engage the AWS DDoS Response Team (DRT) to integrate mitigating controls into the service.

AWS Shield Advanced is a managed service designed to protect applications against external threats like DDoS attacks. It provides enhanced detection and mitigation capabilities, including access to the AWS DDoS Response Team (DRT) who can assist with complex attacks. This option also fulfills the minimal configuration change requirement.

- ⊗ Create an Amazon CloudFront distribution for the application, and set the ALB as the origin. Enable an AWS WAF web ACL on the distribution, and configure rules to block traffic from unknown sources.

While using CloudFront can help with caching and distribution, the WAF configuration issue remains the same as in Option A. The random IP addresses of the DDoS attacks make it difficult to define effective WAF rules.

Overall Explanation:

AWS Shield Advanced is specifically designed for DDoS protection, providing enhanced detection, mitigation, and access to the AWS DDoS Response Team (DRT). This

combination addresses the need for minimal configuration changes and an audit trail of DDoS sources, making it the most suitable solution. Amazon Inspector is an automated vulnerability management service whereas AWS Shield Advanced is a managed service that helps you protect your application against external threats, like DDoS attacks, volumetric bots, and vulnerability exploitation attempts. For higher levels of protection against attacks.

[View Discussions](#)[Ask AI](#)

Question 41

Not Attempted

Storage



A company transfers 200 TB of ocean survey data onto AWS Snowball Edge Storage Optimized devices. The company's high-performance computing (HPC) cluster, hosted on AWS, analyzes the data for oil and gas deposits. A solutions architect requires a solution that provides the HPC cluster with consistent sub-millisecond latency and high-throughput access to the data after the Snowball Edge devices are returned to AWS. Which solution meets these requirements?

- ☐ Create an Amazon S3 bucket. Import the data into the S3 bucket. Configure an AWS Storage Gateway file gateway to use the S3 bucket. Access the file gateway from the HPC cluster instances.
Using Storage Gateway introduces an additional layer that may not meet the sub-millisecond latency requirement.
- ☒ Create an Amazon S3 bucket. Import the data into the S3 bucket. Configure an Amazon FSx for Lustre file system and integrate it with the S3 bucket. Access the FSx for Lustre file system from the HPC cluster instances.
This approach leverages FSx for Lustre, which is designed for high-performance workloads and can provide the required latency and throughput. Snowball integrates with S3, and FSx for Lustre integrates with S3 as a data repository.
- ☐ Create an Amazon S3 bucket and an Amazon Elastic File System (Amazon EFS) file system. Import the data into the S3 bucket. Copy the data from the S3 bucket to the EFS file system. Access the EFS file system from the HPC cluster instances.
EFS is not designed for HPC workloads requiring sub-millisecond latency.
- ☐ Create an Amazon FSx for Lustre file system. Import the data directly into the FSx for Lustre file system. Access the FSx for Lustre file system from the HPC cluster instances.

Direct import from Snowball Edge to FSx for Lustre is not a standard process; data must be imported to S3 first.

Overall Explanation:

The correct solution is to import the data from the Snowball Edge devices into an Amazon S3 bucket and then integrate that S3 bucket with an Amazon FSx for Lustre file system. FSx for Lustre is specifically designed for HPC workloads and provides the necessary sub-millisecond latency and high throughput. Snowball Edge integrates with S3, and FSx for Lustre integrates with S3 as a data repository. Alternatives using Storage Gateway or EFS will not provide the required performance.

[View Discussions](#)[Ask AI](#)

Question 42

Not Attempted

Cost Optimization



A company needs to periodically back up small amounts of data from its on-premises NFS servers to Amazon S3. Which solution is the MOST cost-effective?

- ☐ Set up AWS Glue to copy the data from the on-premises servers to Amazon S3.
AWS Glue is primarily an ETL service and not optimized for simple data transfer, making it less cost-effective for this scenario.
- ☒ Set up an AWS DataSync agent on the on-premises servers and sync the data to Amazon S3.
AWS DataSync is designed for efficient and cost-effective data transfer between on-premises storage and AWS. It is a good fit for periodic backups.
- ☐ Set up an SFTP sync using AWS Transfer for SFTP to sync data from on premises to Amazon S3.
AWS Transfer for SFTP is suitable for SFTP-based transfers, but it can be overkill and more expensive for simple periodic backups.
- ☐ Set up an AWS Direct Connect connection between the on-premises data center and a VPC, and copy the data to Amazon S3.
AWS Direct Connect provides a dedicated network connection, which is likely unnecessary and not cost-effective for transferring small amounts of data periodically. It has a high setup cost.

Network Load Balancers (NLB) are ideal for handling UDP traffic at scale with low latency and are more cost-effective than other options like Gateway Load Balancers. Application Load Balancers do not support UDP. Distributing instances across regions increases cost without directly addressing the need for low latency and high throughput within a region. Therefore, configuring an NLB with the required protocol and ports and specifying the EC2 instances as targets is the best solution.

[View Discussions](#)[Ask AI](#)

Question 44

Not Attempted

Databases



A company runs a three-tier application in a VPC. The database tier uses an Amazon RDS for MySQL DB instance. The company plans to migrate the RDS for MySQL DB instance to an Amazon Aurora PostgreSQL DB cluster. The company needs a solution that replicates the data changes that happen during the migration to the new database. Which combination of steps will meet these requirements?

- ☒ Use AWS Database Migration Service (AWS DMS) Schema Conversion to transform the database objects.
AWS DMS Schema Conversion helps in converting the database schema from MySQL to PostgreSQL, which is a necessary step in migrating between different database engines.
- ☐ Use AWS Database Migration Service (AWS DMS) Schema Conversion to create an Aurora PostgreSQL read replica on the RDS for MySQL DB instance.
Schema Conversion doesn't create Aurora PostgreSQL read replicas.
- ☐ Configure an Aurora MySQL read replica for the RDS for MySQL DB instance.
The company wants to migrate to Aurora PostgreSQL, not Aurora MySQL. This option replicates to Aurora MySQL, which is incorrect.
- ☒ Define an AWS Database Migration Service (AWS DMS) task with change data capture (CDC) to migrate the data.
A DMS task with CDC captures ongoing changes from the source MySQL database and applies them to the target Aurora PostgreSQL database, ensuring data replication during the migration.
- ☐ Promote the Aurora PostgreSQL read replica to a standalone Aurora PostgreSQL DB cluster when the replica lag is zero.

While using Aurora Read Replica is a migration strategy within the same engine family, this option relies on a single step rather than a combination and doesn't address the schema conversion needed for cross-engine migration.

Overall Explanation:

The correct solution involves two key steps: first, using AWS DMS Schema Conversion to transform the database objects from MySQL to PostgreSQL format, and second, using a DMS task with Change Data Capture (CDC) to replicate the data changes from the source MySQL database to the target Aurora PostgreSQL database during the migration. This ensures minimal downtime and data consistency. Option 2 is incorrect because schema conversion does not create read replicas. Option 3 proposes migrating to Aurora MySQL, which is not the desired target. Option 5 is not a valid approach for this particular migration scenario.

[View Discussions](#)[Ask AI](#)

Question 45

Not Attempted

Databases



A company hosts a database that runs on an Amazon RDS instance deployed to multiple Availability Zones. The company periodically runs a script against the database to report new entries. The script negatively affects the performance of a critical application. The company needs to improve application performance with minimal cost and operational overhead. Which solution meets these requirements?

- ☐ Add functionality to the script to identify the instance that has the fewest active connections. Configure the script to read from that instance to report the total new entries.

While this might distribute the load, it adds complexity to the script and doesn't completely isolate the reporting workload.

- ☒ Create a read replica of the database. Configure the script to query only the read replica to report the total new entries.

This is the best solution. Read replicas are designed for read-heavy workloads like reporting. This offloads the reporting workload from the primary database, improving application performance without significant cost or operational overhead.

- ⊗ Instruct the development team to manually export the new entries for the day in the database at the end of each day.

This is not a good solution. Manual exports are time-consuming, error-prone, and increase operational overhead.

- ⊗ Use Amazon ElastiCache to cache the common queries that the script runs against the database.

ElastiCache helps improve the performance of frequently accessed data but doesn't address the fundamental issue of the reporting script impacting database performance. The script still runs, potentially affecting the application.

Overall Explanation:

Creating a read replica and directing the reporting script to query only the read replica is the most effective solution. It offloads the reporting workload from the primary database, minimizing the impact on application performance while maintaining low cost and minimal operational overhead. Other options are either too complex, require manual intervention, or don't address the underlying problem effectively.

 View Discussions

 Ask AI

Question 46

Not Attempted

Monitoring and Observability



A company is using an Application Load Balancer (ALB) to present its application to the internet. The company is experiencing abnormal traffic patterns. A solutions architect needs to improve visibility into the infrastructure to understand these anomalies in a cost-effective and operationally efficient manner. Which solution meets these requirements?

- ⊗ Create a table in Amazon Athena for AWS CloudTrail logs. Create a query for the relevant information.

CloudTrail logs API calls and changes on AWS resources, but it does not log every traffic access pattern. This option is not the most efficient for analyzing traffic patterns.

- ✔ Enable ALB access logging to Amazon S3. Create a table in Amazon Athena, and query the logs.

ALB access logs provide detailed information about requests received by the ALB. Using Amazon Athena to query these logs in S3 is an efficient way to analyze traffic patterns.

- ⊗ Enable ALB access logging to Amazon S3. Open each file in a text editor, and search each line for the relevant information.

Manually opening and searching log files is not operationally efficient, especially when dealing with a large volume of logs.

- ⊗ Use Amazon EMR on a dedicated Amazon EC2 instance to directly query the ALB to acquire traffic access log information.

Using Amazon EMR is an overkill. Athena is more suitable and cost effective for querying logs in S3.

Overall Explanation:

Enabling ALB access logs to Amazon S3 and querying them with Amazon Athena is the most operationally efficient solution. ALB access logs provide the necessary traffic information. Athena allows for querying the data directly in S3 using standard SQL, providing detailed visibility into traffic patterns without requiring complex infrastructure management or manual analysis.

[View Discussions](#)[Ask AI](#)

Question 47

Not Attempted

Networking



A company needs to allow its Amazon EC2 instances in private subnets to connect to the public internet for software updates and package installations, without allowing inbound connections from the internet. The company wants to use NAT gateways to achieve this. Which of the following solutions will meet the company's requirements?

- ⊗ Create public NAT gateways in the same private subnets as the EC2 instances.

This is incorrect because a NAT gateway needs to be in a public subnet to access the internet. Placing it in a private subnet defeats the purpose of using a NAT gateway for outbound internet access.

- ⊗ Create private NAT gateways in the same private subnets as the EC2 instances.

This is incorrect. Private NAT gateways are used for VPC or on-premises network connectivity, not direct internet access. Traffic from a private NAT gateway cannot be routed directly to an internet gateway.

- ✔ Create public NAT gateways in public subnets in the same VPCs as the EC2 instances.

This is correct. Public NAT gateways reside in public subnets and have an Elastic IP address associated with them, enabling EC2 instances in private subnets to initiate outbound traffic to the internet.

- ⊗ Create private NAT gateways in public subnets in the same VPCs as the EC2 instances.

This is incorrect. While placing the NAT gateway in a public subnet is correct, creating a *private* NAT gateway won't allow direct access to the Internet. Private NAT gateways are intended for communication with other VPCs or on-premises networks via Transit Gateway or Virtual Private Gateway, not directly to the Internet.

Overall Explanation:

To allow EC2 instances in private subnets to connect to the internet, you need to create public NAT gateways in public subnets. The EC2 instances in the private subnets will route their internet-bound traffic to the NAT gateways in the public subnets. The NAT gateways then forward the traffic to the internet, and the responses are routed back to the EC2 instances. Private NAT Gateways are not meant to connect directly to the internet.

 View Discussions

 Ask AI

Question 48

Not Attempted

Security



A company has an organization in AWS Organizations. The company runs Amazon EC2 instances across four AWS accounts in the root organizational unit (OU). There are three nonproduction accounts and one production account. The company wants to prohibit users from launching EC2 instances of a certain size in the nonproduction accounts. The company has created a service control policy (SCP) to deny access to launch instances that use the prohibited types. Which solutions to deploy the SCP will meet these requirements?

- ⊗ Attach the SCP to the root OU for the organization.

Attaching the SCP to the root OU will apply it to all accounts, including the production account, which is not desired.

- ✓ Attach the SCP to the three nonproduction Organizations member accounts.

Attaching the SCP directly to the nonproduction accounts is a valid approach to restrict instance types in those accounts.

- ⊗ Attach the SCP to the Organizations management account.

Attaching the SCP to the management account will not directly enforce the policy on the member accounts. SCPs must be attached to the root, OUs, or accounts themselves.

- ⊗ Create an OU for the production account. Attach the SCP to the OU. Move the production member account into the new OU.

This approach will restrict the production account, which is contrary to the requirement.

- ✓ Create an OU for the required accounts. Attach the SCP to the OU. Move the nonproduction member accounts into the new OU.

Creating an OU for the nonproduction accounts and attaching the SCP to it allows for targeted application of the policy to only the desired accounts.

Overall Explanation:

The two correct solutions are to attach the SCP directly to the three nonproduction accounts or to create a new OU, move the nonproduction accounts into it, and then attach the SCP to the OU. This ensures that only the nonproduction accounts are affected by the SCP, as desired.

 View Discussions

 Ask AI

Question 49

Not Attempted

Databases



A company's shopping application uses Amazon DynamoDB to store customer information. In the event of data corruption, a solutions architect needs to design a solution that meets a recovery point objective (RPO) of 15 minutes and a recovery time objective (RTO) of 1 hour. Which solution should the solutions architect recommend to meet these requirements?

- ⊗ Configure DynamoDB global tables. For RPO recovery, point the application to a different AWS Region.

DynamoDB global tables provide multi-Region, multi-active database functionality but do not inherently address data corruption scenarios requiring rapid recovery. They are more suitable for disaster recovery and high availability across regions, not for granular data recovery within the specified RPO.

- ✓ Configure DynamoDB point-in-time recovery. For RPO recovery, restore to the desired point in time.

DynamoDB point-in-time recovery (PITR) is designed for continuous backup and fast recovery. It allows restoring a table to any point in time within the last 35 days, with granularity down to

seconds, which perfectly aligns with the RPO of 15 minutes. The restoration process can typically be completed within the RTO of 1 hour.

- ⊗ Export the DynamoDB data to Amazon S3 Glacier on a daily basis. For RPO recovery, import the data from S3 Glacier to DynamoDB.

Exporting data to S3 Glacier on a daily basis does not meet the required RPO of 15 minutes. Glacier is designed for long-term archival storage, and restoring data from Glacier can take hours, which exceeds the specified RTO.

- ⊗ Schedule Amazon Elastic Block Store (Amazon EBS) snapshots for the DynamoDB table every 15 minutes. For RPO recovery, restore the DynamoDB table by using the EBS snapshot.

DynamoDB is a serverless database service and does not directly involve Amazon EBS. EBS snapshots are not applicable or supported for backing up and restoring DynamoDB tables. This option is therefore incorrect.

Overall Explanation:

DynamoDB point-in-time recovery (PITR) provides the most straightforward and effective solution for meeting the specified RPO of 15 minutes and RTO of 1 hour. By enabling PITR and restoring the table to the desired point in time, the company can recover the customer information with minimal data loss and within the required time frame. Global tables are more suited for disaster recovery, while S3 Glacier and EBS Snapshots are not suitable for these requirements.

[View Discussions](#)[Ask AI](#)

Question 50

Not Attempted

Networking



A company hosts its website on Amazon EC2 instances. The website processes classified data stored in Amazon S3. Due to stringent security requirements, the company needs to establish a private and secure connection between its EC2 resources and Amazon S3, ensuring data doesn't traverse the public internet. Which solution provides the MOST secure and private connection?

✓ Set up S3 bucket policies to allow access from a VPC endpoint.

A VPC endpoint for S3 allows secure access without using public IPs and without requiring traffic to traverse the internet. Bucket policies can then be used to further restrict access to only come from the VPC endpoint.

✗ Set up an IAM policy to grant read-write access to the S3 bucket.

While IAM policies are essential for controlling access to S3, they do not, by themselves, establish a private connection. They control **who** can access S3 but not **how** they access it.

✗ Set up a NAT gateway to access resources outside the private subnet.

A NAT Gateway allows instances in a private subnet to connect to the internet, but it doesn't provide a private connection to S3. In fact, it directs traffic to the public internet.

✗ Set up an access key ID and a secret access key to access the S3 bucket.

Access keys, like IAM policies, control **who** can access the S3 bucket, not **how**. Using access keys alone doesn't establish a private connection, and keeping them secure is a significant concern.

Overall Explanation:

A VPC endpoint establishes a private connection between your VPC and S3, ensuring data doesn't leave the AWS network and is not exposed to the public internet. S3 bucket policies, configured in conjunction with the VPC endpoint, further restrict access to only requests originating from within your VPC, enhancing security. Therefore, setting up S3 bucket policies to allow access from a VPC endpoint provides the MOST secure and private connection.

 View Discussions

 Ask AI

Question 51

Not Attempted

Databases



An ecommerce company's application, running on AWS, utilizes an Amazon Aurora PostgreSQL cluster in Multi-AZ mode for its database. During a recent promotional campaign, the application experienced heavy read and write loads, leading to user timeouts. A solutions architect needs to improve the application's scalability and high availability with minimal downtime. Which solution best meets these requirements?

- ⊗ Create an Amazon EventBridge rule with the Aurora cluster as a source. Configure an AWS Lambda function to log the state change events of the Aurora cluster and add the Lambda function as a target for the EventBridge rule. Add additional reader nodes to fail over to.

This option focuses on logging state changes but does not directly address the read/write load or connection issues.

- ⊗ Modify the Aurora cluster and activate the zero-downtime restart (ZDR) feature. Use Database Activity Streams on the cluster to track the cluster status.

ZDR is useful for automated restarts for error conditions but does not address scalability or connection management during high load.

- ✓ Add additional reader instances to the Aurora cluster. Create an Amazon RDS Proxy target group for the Aurora cluster.

Adding reader instances helps offload read workloads from the primary instance, improving read scalability. RDS Proxy manages database connections, reducing timeouts and improving overall application performance.

- ⊗ Create an Amazon ElastiCache for Redis cache. Replicate data from the Aurora cluster to Redis by using AWS Database Migration Service (AWS DMS) with a write-around approach.

Using ElastiCache with a write-around approach might not address the immediate issue of heavy write load on the database and introduces eventual consistency issues.

Overall Explanation:

Option C is the most effective solution. Adding Aurora Replicas will help offload read workloads from the primary DB instance. Amazon RDS Proxy allows applications to pool and share connections established with the database, improving database efficiency and application scalability, thereby minimizing downtime.

 View Discussions

 Ask AI

Question 52

Not Attempted

Networking



A company is designing a web application on AWS. The application will use a VPN connection between the company's existing data centers and the company's VPCs. The company uses Amazon Route 53 as its DNS service. The application must use private DNS records to

communicate with the on-premises services from a VPC. Which solution will meet these requirements in the MOST secure manner?

- ☒ Create a Route 53 Resolver outbound endpoint. Create a resolver rule. Associate the resolver rule with the VPC.

An outbound resolver endpoint forwards DNS queries from your VPC to your on-premises DNS service. A resolver rule specifies the domain names for the DNS queries that you want to forward (such as example.com), and the IP addresses of the DNS resolvers in your on-premises network. This allows the application in the VPC to resolve domain names hosted on-premises.

- ☐ Create a Route 53 Resolver inbound endpoint. Create a resolver rule. Associate the resolver rule with the VPC.

An inbound resolver endpoint allows DNS queries to your VPC from your on-premises network. This setup is for allowing on-premises resources to resolve DNS queries within the VPC, not the other way around.

- ☐ Create a Route 53 private hosted zone. Associate the private hosted zone with the VPC.

While private hosted zones are for private DNS records within a VPC, creating only a private hosted zone is not sufficient to resolve on-premise resources from within the VPC if the DNS records for on-premise are not stored in route53.

- ☐ Create a Route 53 public hosted zone. Create a record for each service to allow service communication

Public hosted zones are for publicly accessible domain names and records, which does not meet the requirement for private communication with on-premises services. Also, it wouldn't be secure to expose on-premise services on public dns.

Overall Explanation:

The correct solution involves creating a Route 53 Resolver outbound endpoint and a resolver rule associated with the VPC. This configuration enables DNS queries from the VPC to be forwarded to the on-premises DNS servers, allowing the application to resolve private DNS records for services located in the on-premises data centers. Inbound resolver is for resolving VPC dns from on-premise, while public zones are for public dns records.

 View Discussions

 Ask AI

A company is running a photo hosting service in the us-east-1 Region. The service enables users across multiple countries to upload and view photos. Some photos are heavily viewed for months, and others are viewed for less than a week. The application allows uploads of up to 20 MB for each photo. The service uses the photo metadata to determine which photos to display to each user. Which solution provides the appropriate user access MOST cost-effectively?

- ☐ Store the photos in Amazon DynamoDB. Turn on DynamoDB Accelerator (DAX) to cache frequently viewed items.

Storing photos directly in DynamoDB is not cost-effective for large files. DAX would help with caching frequently viewed items, but DynamoDB itself is more expensive than object storage.

- ☒ Store the photos in the Amazon S3 Intelligent-Tiering storage class. Store the photo metadata and its S3 location in DynamoDB.

Amazon S3 Intelligent-Tiering automatically moves objects between frequent and infrequent access tiers based on access patterns, optimizing costs. Storing metadata in DynamoDB allows for efficient querying and retrieval of photo information.

- ☐ Store the photos in the Amazon S3 Standard storage class. Set up an S3 Lifecycle policy to move photos older than 30 days to the S3 Standard-Infrequent Access (S3 Standard-IA) storage class. Use the object tags to keep track of metadata.

While using S3 Standard and S3 Standard-IA is cost-effective, it requires manually setting a lifecycle policy. It doesn't automatically optimize storage costs based on access patterns like Intelligent-Tiering. Also, storing all metadata as object tags is less efficient than using DynamoDB for querying and indexing.

- ☐ Store the photos in the Amazon S3 Glacier storage class. Set up an S3 Lifecycle policy to move photos older than 30 days to the S3 Glacier Deep Archive storage class. Store the photo metadata and its S3 location in Amazon OpenSearch Service.

Storing photos in S3 Glacier would be very cheap but not suitable for viewing photos, as the retrieval process from Glacier takes a significant amount of time and cost to retrieve. Storing metadata in Amazon OpenSearch Service is overkill and more complex than using DynamoDB.

Overall Explanation:

The most cost-effective solution is to store the photos in Amazon S3 Intelligent-Tiering and store the metadata in DynamoDB. S3 Intelligent-Tiering automatically manages storage costs based on the frequency of access, while DynamoDB provides efficient metadata management for searching and displaying photos. Options A, C, and D are less efficient in terms of cost or functionality for this use case.

[View Discussions](#)[Ask AI](#)

Question 54

Not Attempted

Monitoring and Observability



A company runs a highly available web application on Amazon EC2 instances behind an Application Load Balancer. The company uses Amazon CloudWatch metrics. As traffic to the web application increases, some EC2 instances become overloaded with many outstanding requests. CloudWatch metrics show that the number of requests processed and the time to receive responses from some EC2 instances are both higher compared to other EC2 instances. The company wants to prevent new requests from being forwarded to the overloaded EC2 instances. Which solution will meet these requirements?

- ☐ Use the round robin routing algorithm based on the RequestCountPerTarget and ActiveConnectionCount CloudWatch metrics.
Round robin distributes requests equally, without considering the current load on each instance. Therefore, it doesn't prevent overloading instances.
- ☒ Use the least outstanding requests algorithm based on the RequestCountPerTarget and ActiveConnectionCount CloudWatch metrics.
The least outstanding requests algorithm, combined with RequestCountPerTarget and ActiveConnectionCount, will route requests to targets with the lowest number of in-progress requests, helping to avoid overloading instances.
- ☐ Use the round robin routing algorithm based on the RequestCount and TargetResponseTime CloudWatch metrics.
Round robin distributes requests equally, irrespective of instance load. Using RequestCount (for the entire ALB) and TargetResponseTime wouldn't effectively distribute load based on individual instance capacity.
- ☐ Use the least outstanding requests algorithm based on the RequestCount and TargetResponseTime CloudWatch metrics.
While least outstanding requests is a good approach, using RequestCount (ALB level) is not granular enough. TargetResponseTime is a good indicator but not the best for directly controlling request distribution based on outstanding requests.

Overall Explanation:

The correct solution is to use the least outstanding requests algorithm based on the RequestCountPerTarget and ActiveConnectionCount CloudWatch metrics. The 'least

outstanding requests' algorithm routes traffic to instances with the fewest active requests, preventing overloading. RequestCountPerTarget measures the number of requests for each target, and ActiveConnectionCount indicates the current number of active TCP connections, complementing the request count to identify overwhelmed instances. Round robin does not consider the existing load, and using RequestCount (ALB level) wouldn't distribute the load effectively based on individual instance capacity.

[View Discussions](#)[Ask AI](#)

Question 55

Not Attempted

Cost Optimization



A company uses Amazon EC2, AWS Fargate, and AWS Lambda to run multiple workloads in the company's AWS account. The company wants to fully make use of its Compute Savings Plans. The company wants to receive notification when coverage of the Compute Savings Plans drops. Which solution will meet these requirements with the MOST operational efficiency?

- ☒ Create a daily budget for the Savings Plans by using AWS Budgets. Configure the budget with a coverage threshold to send notifications to the appropriate email message recipients.
This is the most operationally efficient solution. AWS Budgets allows you to set a coverage threshold and receive notifications when coverage drops below the defined level. This avoids custom coding and maintenance.
- ☐ Create a Lambda function that runs a coverage report against the Savings Plans. Use Amazon Simple Email Service (Amazon SES) to email the report to the appropriate email message recipients.
This solution requires custom code, increasing operational overhead. While feasible, it is not the most efficient.
- ☐ Create an AWS Budgets report for the Savings Plans budget. Set the frequency to daily.
This option creates a report, but it does not automatically send notifications when the coverage drops. It requires manual review of the report.
- ☐ Create a Savings Plans alert subscription. Enable all notification options. Enter an email address to receive notifications.
While Savings Plan alerts exist, using AWS Budgets for coverage alerts is a more established and versatile method for tracking and receiving notifications on Savings Plans utilization and coverage.

Overall Explanation:

AWS Budgets provides a built-in mechanism to monitor Savings Plans coverage and send notifications when the coverage falls below a defined threshold. This solution is the most operationally efficient because it leverages existing AWS services and avoids custom development. Using AWS Budgets allows you to set a coverage threshold and receive notifications when coverage drops below the defined level.

[View Discussions](#)[Ask AI](#)

Question 56

Not Attempted

Networking



A company runs a real-time data ingestion solution on AWS using Amazon Managed Streaming for Apache Kafka (Amazon MSK). The MSK cluster is deployed in a VPC in private subnets across three Availability Zones. The solution needs to be redesigned to be publicly available over the internet while ensuring the data in transit is encrypted. Which solution will meet these requirements with the MOST operational efficiency?

- ☒ **Configure public subnets in the existing VPC. Deploy an MSK cluster in the public subnets. Update the MSK cluster security settings to enable mutual TLS authentication.**
This option allows the MSK cluster to be publicly accessible and encrypts data in transit using mutual TLS. It leverages the existing VPC, minimizing operational overhead.
- ☐ **Create a new VPC that has public subnets. Deploy an MSK cluster in the public subnets. Update the MSK cluster security settings to enable mutual TLS authentication.**
Creating a new VPC adds unnecessary complexity and operational overhead. Using the existing VPC is more efficient.
- ☐ **Deploy an Application Load Balancer (ALB) that uses private subnets. Configure an ALB security group inbound rule to allow inbound traffic from the VPC CIDR block for HTTPS protocol.**
ALBs are not suitable for handling real-time data ingestion scenarios, especially with protocols like TCP. Furthermore, Kafka has its own load balancing mechanisms.
- ☐ **Deploy a Network Load Balancer (NLB) that uses private subnets. Configure an NLB listener for HTTPS communication over the internet.**
NLBs are not designed to handle HTTPS traffic directly, and this approach doesn't leverage Kafka's built-in load balancing capabilities. Also, using private subnets defeats the purpose of making the

solution publicly accessible.

Overall Explanation:

The most operationally efficient solution is to configure public subnets in the existing VPC and deploy the MSK cluster there. Enabling mutual TLS authentication ensures the data in transit is encrypted. This option avoids creating new VPCs or using load balancers unnecessarily, simplifying the architecture and management. Kafka is capable of load balancing itself and can handle the real-time data ingestion requirements effectively.

[View Discussions](#)[Ask AI](#)**Question 57**

Not Attempted

Serverless



A company is migrating a legacy on-premises application to AWS. This application ingests customer order files from an on-premises enterprise resource planning (ERP) system and uploads them to an SFTP server. The application currently uses a scheduled job, executed hourly, to check for order files. The company's AWS account has existing connectivity to the on-premises network. The new application on AWS must integrate with the existing ERP system, be secure and resilient, and process orders from the ERP system using the SFTP protocol immediately upon arrival. Which solution best meets these requirements?

- ☒ Create an AWS Transfer Family SFTP internet-facing server in two Availability Zones. Use Amazon S3 storage. Create an AWS Lambda function to process order files. Use S3 Event Notifications to send `s3:ObjectCreated:*` events to the Lambda function.

This option uses an internet-facing server, which is less secure than an internal server given the existing on-premises connectivity. It also relies on S3 Event Notifications, which are not necessary when Transfer Family Managed Workflows can directly invoke Lambda.

- ☒ Create an AWS Transfer Family SFTP internet-facing server in one Availability Zone. Use Amazon Elastic File System (Amazon EFS) storage. Create an AWS Lambda function to process order files. Use a Transfer Family managed workflow to invoke the Lambda function.

This option uses an internet-facing server, which is less secure than an internal server given the existing on-premises connectivity. Also, using EFS is not optimal for this use case and only provides single AZ resiliency.

- ⊗ Create an AWS Transfer Family SFTP internal server in two Availability Zones. Use Amazon Elastic File System (Amazon EFS) storage. Create an AWS Step Functions state machine to process order files. Use Amazon EventBridge Scheduler to invoke the state machine to periodically check Amazon EFS for order files.

This option uses Amazon EFS, which is less suitable for immediate processing of files compared to S3. Additionally, using Amazon EventBridge Scheduler negates the requirement for immediate processing upon file arrival.

- ✓ Create an AWS Transfer Family SFTP internal server in two Availability Zones. Use Amazon S3 storage. Create an AWS Lambda function to process order files. Use a Transfer Family managed workflow to invoke the Lambda function.

This option provides a secure and resilient solution using an internal SFTP server within two Availability Zones. It uses Amazon S3 for storage, which is efficient and scalable. Transfer Family managed workflows enable immediate invocation of a Lambda function upon file arrival, fulfilling the requirement for immediate processing.

Overall Explanation:

The best solution is to create an AWS Transfer Family SFTP internal server in two Availability Zones with Amazon S3 storage. This is the most secure option as it leverages the existing connection between the on-premises environment and AWS. Using an internal server eliminates the need for public internet access. The Transfer Family managed workflow triggers a Lambda function immediately after a file is uploaded to S3, meeting the requirement to process orders immediately. S3 is preferred over EFS because it integrates seamlessly with Lambda via Transfer Family workflows and is optimized for object storage.

 View Discussions

 Ask AI

Question 58

Not Attempted

Compute



A company's applications use Apache Hadoop and Apache Spark to process data on premises. The existing infrastructure is not scalable and is complex to manage. A solutions architect must design a scalable solution that reduces operational complexity. The solution must keep the data processing on premises. Which solution will meet these requirements?

- ⊗ Use AWS Site-to-Site VPN to access the on-premises Hadoop Distributed File System (HDFS) data and application. Use an Amazon EMR cluster to process the data.

This option involves moving data processing to the cloud, which does not satisfy the requirement to keep data processing on premises.

- ⊗ Use AWS DataSync to connect to the on-premises Hadoop Distributed File System (HDFS) cluster. Create an Amazon EMR cluster to process the data.

This option also involves moving data processing to the cloud, which does not satisfy the requirement to keep data processing on premises.

- ✓ Migrate the Apache Hadoop application and the Apache Spark application to Amazon EMR clusters on AWS Outposts. Use the EMR clusters to process the data.

This solution keeps the data processing on premises using AWS Outposts, which brings AWS infrastructure and services to the company's data center. EMR provides a managed Hadoop and Spark environment, reducing operational complexity and enabling scalability.

- ⊗ Use an AWS Snowball device to migrate the data to an Amazon S3 bucket. Create an Amazon EMR cluster to process the data.

This option involves moving data and processing to the cloud (S3), which does not satisfy the requirement to keep data processing on premises.

Overall Explanation:

The best solution is to migrate the Apache Hadoop and Spark applications to Amazon EMR clusters on AWS Outposts. This approach satisfies the requirement to keep data processing on premises while providing a scalable, managed environment, thus reducing operational complexity. Options A, B, and D involve processing the data in the cloud, which does not meet the stated requirement.

 View Discussions

 Ask AI

Question 59

Not Attempted

Storage



A company is migrating a large amount of data from on-premises storage to AWS. Windows, Mac, and Linux-based Amazon EC2 instances in the same AWS Region will access the data by using SMB and NFS storage protocols. The company will access a portion of the data routinely. The company will access the remaining data infrequently. The company needs to design a solution to host the data. Which solution will meet these requirements with the LEAST operational overhead?

- ⊗ Create an Amazon Elastic File System (Amazon EFS) volume that uses EFS Intelligent-Tiering. Use AWS DataSync to migrate the data to the EFS volume.

Amazon EFS does not natively support SMB protocol. This solution would require additional configuration and overhead to support Windows-based instances.

- ✔ Create an Amazon FSx for ONTAP instance. Create an FSx for ONTAP file system with a root volume that uses the auto tiering policy. Migrate the data to the FSx for ONTAP volume.

Amazon FSx for ONTAP supports both SMB and NFS protocols, along with automatic tiering for cost optimization. It offers a fully managed solution with minimal operational overhead.

- ⊗ Create an Amazon S3 bucket that uses S3 Intelligent-Tiering. Migrate the data to the S3 bucket by using an AWS Storage Gateway Amazon S3 File Gateway.

While S3 Intelligent-Tiering addresses infrequent access, using Storage Gateway adds operational overhead for file access from EC2 instances within the same AWS Region. Storage Gateway is better suited for connecting on-premises environments to S3.

- ⊗ Create an Amazon FSx for OpenZFS file system. Migrate the data to the new volume.

Amazon FSx for OpenZFS does not offer built-in tiering for infrequently accessed data. This solution may result in higher storage costs.

Overall Explanation:

Amazon FSx for ONTAP provides a fully managed file system that supports both SMB and NFS protocols, essential for Windows, Mac, and Linux instances. The auto-tiering policy automatically moves infrequently accessed data to lower-cost storage tiers, minimizing operational overhead and storage costs. Therefore, option B is the most suitable solution.

 View Discussions

 Ask AI

Question 60

Not Attempted

Cost Optimization



A company's application, running within an Amazon VPC, frequently uploads and downloads images from Amazon S3 buckets located in the same AWS Region. The solutions architect observes high data transfer costs and seeks a solution to minimize these expenses. How should the solutions architect reduce these data transfer costs?

- ⊗ Deploy Amazon API Gateway into a public subnet and route S3 calls through it.

Using API Gateway as a proxy for S3 requests adds complexity and incurs unnecessary costs compared to using a VPC endpoint directly.

- ⊗ Deploy a NAT gateway into a public subnet and attach an endpoint policy to allow access to the S3 buckets.

Using a NAT gateway for accessing S3 introduces unnecessary data transfer costs as traffic still flows over the internet.

- ⊗ Deploy the application into a public subnet and route traffic through an internet gateway to access the S3 buckets.

This approach will incur data transfer fees as the traffic goes through the public internet.

- ✓ Deploy an S3 VPC gateway endpoint into the VPC and attach an endpoint policy to allow access to the S3 buckets.

An S3 VPC gateway endpoint provides a direct and cost-effective solution for accessing S3 buckets within the same Region, keeping data transfer within the AWS network infrastructure and reducing fees. It also allows secure access to the S3 resources.

Overall Explanation:

The optimal solution is to deploy an S3 VPC gateway endpoint within the VPC. This facilitates private connections between VPCs and S3 without traversing the public internet, thereby avoiding data transfer fees for traffic between the VPC and S3 within the same region. The endpoint policy controls access to specific S3 buckets, enhancing security.

 View Discussions

 Ask AI

Question 61

Not Attempted

Serverless



A manufacturing company's report generation application, currently a monolith on a single Amazon EC2 instance, takes 20 minutes per report. The application requires frequent updates to tightly coupled modules, making maintenance complex and causing downtime with each patch. Report generation restarts from the beginning after interruptions. The company seeks a flexible, scalable, and incrementally improvable solution that minimizes downtime. Which of the following options BEST addresses these requirements?

- ⊗ Run the application on AWS Lambda as a single function with maximum provisioned concurrency.

Lambda functions have execution time limits. Running a 20-minute report generation process as a single Lambda function might exceed these limits. Also, a single Lambda function would not provide the desired modularity.

- ⊗ Run the application on Amazon EC2 Spot Instances as microservices with a Spot Fleet default allocation strategy.

While Spot Instances can be cost-effective, their potential for interruption makes them unsuitable for long-running, stateful tasks like report generation, especially given the requirement to restart from the beginning after interruptions. Furthermore, while the microservices architecture is a good direction, the spot fleet strategy doesn't address the reliability needed.

- ✓ Run the application on Amazon Elastic Container Service (Amazon ECS) as microservices with service auto scaling.

This is the most suitable option. Decomposing the application into microservices hosted on ECS allows for independent deployments and scaling. Service auto scaling ensures that resources are available to handle varying workloads. This approach enhances flexibility, scalability, and reduces downtime during updates.

- ⊗ Run the application on AWS Elastic Beanstalk as a single application environment with an all-at-once deployment strategy.

Using Elastic Beanstalk in a single environment with an all-at-once deployment strategy replicates the existing downtime problem. While Elastic Beanstalk simplifies deployment, it doesn't address the underlying monolithic architecture's limitations.

Overall Explanation:

The BEST solution is to refactor the application into microservices and run them on Amazon ECS with service auto scaling. This approach provides the desired flexibility, scalability, and reduced downtime. Option A is incorrect because Lambda functions have execution time limits. Option B is incorrect because Spot Instances are prone to interruption. Option D is incorrect because it uses an all-at-once deployment, which leads to downtime. Therefore, ECS is the only option that meets all requirements.

 View Discussions

 Ask AI

Question 62

Not Attempted

Serverless



A company is refactoring a monolithic Python web application into a serverless microservices architecture on AWS. They've selected a component that handles hundreds of requests per

second to test as a microservice. The solution must automatically scale, require minimal infrastructure management, and effectively support Python. Which AWS service is MOST suitable for deploying and testing this microservice component?

- ☐ Use a Spot Fleet with auto scaling of EC2 instances running Amazon Linux.
This option involves managing EC2 instances, which contradicts the requirement for minimal infrastructure management. Spot Fleets are also not ideal for consistent, high-volume request handling.
- ☐ Use an AWS Elastic Beanstalk web server environment with high availability configured.
Elastic Beanstalk, while simplifying deployment, still requires managing an environment. It is also overkill for testing a single microservice component and introduces unnecessary complexity.
- ☐ Use Amazon Elastic Kubernetes Service (Amazon EKS) and launch Auto Scaling groups of self-managed EC2 instances.
EKS provides container orchestration, but self-managing EC2 instances goes against the requirement of minimal infrastructure management. EKS also adds complexity, as it requires configuring and managing Kubernetes clusters.
- ☒ Use an AWS Lambda function with custom-developed code.
AWS Lambda is a serverless compute service that supports Python and automatically scales based on request volume. It requires minimal infrastructure management, making it ideal for testing a microservice component.

Overall Explanation:

AWS Lambda provides a serverless, scalable, and Python-compatible environment perfectly suited for testing the microservice component. It aligns with the requirements of minimal infrastructure management and automatic scaling to handle hundreds of requests per second. Spot Fleets, Elastic Beanstalk, and EKS all involve more infrastructure management and complexity, making Lambda the most appropriate choice for this scenario.

[View Discussions](#)[Ask AI](#)

Question 63

Not Attempted

Networking



A company connects its on-premises network to an AWS account using AWS Direct Connect. This account hosts 30 VPCs within the same AWS Region. Each VPC uses private virtual

interfaces (VIFs) and has a unique, non-overlapping CIDR block. The company aims to centralize network management while enabling seamless communication between all VPCs and the on-premises network with minimal operational overhead. Which solution achieves this?

- ☒ Create a transit gateway, associate the Direct Connect connection with a new transit VIF, and enable route propagation on the transit gateway.

This is the most efficient solution. A transit gateway acts as a central hub, simplifying routing between the Direct Connect connection and all associated VPCs. Route propagation automatically updates routes within the transit gateway.

- ☐ Create a Direct Connect gateway. Recreate the private VIFs to use the new gateway. Associate each VPC by creating new virtual private gateways.

This involves more configuration and potential disruption by requiring recreation of the private VIFs. Managing virtual private gateways for each VPC also adds to the operational overhead.

- ☐ Create a transit VPC. Connect the Direct Connect connection to the transit VPC. Create a peering connection between all other VPCs in the Region. Update the route tables.

While possible, creating peering connections between all VPCs creates a complex, fully meshed network that is difficult to manage and scale. It also requires manual route table updates.

- ☐ Create AWS Site-to-Site VPN connections from on premises to each VPC. Ensure that both VPN tunnels are UP for each connection. Turn on the route propagation feature.

Creating VPN connections to each VPC significantly increases operational overhead, and managing multiple VPN connections can be complex and costly.

Overall Explanation:

The optimal solution is to use a transit gateway. This provides a centralized hub for network traffic, streamlining communication between the on-premises network (via Direct Connect) and all 30 VPCs. By associating the Direct Connect connection with a transit VIF and enabling route propagation, the solution minimizes operational overhead and ensures seamless connectivity. Direct Connect gateway does not allow connectivity between VPC's, a Transit Gateway is needed.

 View Discussions

 Ask AI

Question 64

Not Attempted

Compute



A company has applications that run on Amazon EC2 instances. The EC2 instances connect to Amazon RDS databases by using an IAM role that has associated policies. The company wants to use AWS Systems Manager to patch the EC2 instances without disrupting the running applications. Which solution will meet these requirements?

- ☐ Create a new IAM role. Attach the AmazonSSMManagedInstanceCore policy to the new IAM role. Attach the new IAM role to the EC2 instances and the existing IAM role.

This option involves creating a new IAM role, which can be a valid approach, but may require more manual configuration and management than necessary. It doesn't inherently guarantee non-disruption of existing application access to RDS.

- ☐ Create an IAM user. Attach the AmazonSSMManagedInstanceCore policy to the IAM user. Configure Systems Manager to use the IAM user to manage the EC2 instances.

Using an IAM user is generally not recommended for managing EC2 instances. IAM roles are preferred for this purpose. This approach also doesn't integrate well with the existing IAM role setup.

- ☒ Enable Default Host Configuration Management in Systems Manager to manage the EC2 instances.

Enabling Default Host Configuration Management allows Systems Manager to automatically manage the EC2 instances with minimal configuration. It creates and applies a default IAM role, ensuring Systems Manager has the necessary permissions without disrupting the existing IAM role used for database access. This simplifies the process and is the best option to avoid disrupting applications.

- ☐ Remove the existing policies from the existing IAM role. Add the AmazonSSMManagedInstanceCore policy to the existing IAM role.

Removing existing policies from the IAM role will disrupt the applications' access to the RDS databases. This directly contradicts the requirement to avoid disruption.

Overall Explanation:

Enabling Default Host Configuration Management in Systems Manager is the optimal solution. It allows Systems Manager to manage the instances without requiring manual IAM role creation or modification of the existing role, thus preventing any disruption to the applications' database access. The Default Host Management Configuration setting allows AWS Systems Manager to manage your Amazon EC2 instances automatically as managed instances.

[View Discussions](#)[Ask AI](#)

Question 65

Not Attempted

[Compute](#)

A company runs container applications using Amazon Elastic Kubernetes Service (Amazon EKS) and the Kubernetes Horizontal Pod Autoscaler. The workload fluctuates throughout the day. A solutions architect observes that the number of nodes doesn't automatically scale out when the existing nodes reach maximum capacity, leading to performance issues in the cluster. Which solution will resolve this issue with the LEAST administrative overhead?

- ☐ ☒ Scale out the nodes by tracking the memory usage.
Scaling based solely on memory usage is insufficient. It doesn't address the core issue of automatic node scaling within the Kubernetes cluster.
- ☒ Use the Kubernetes Cluster Autoscaler to manage the number of nodes in the cluster.
The Kubernetes Cluster Autoscaler automatically adjusts the number of nodes in the cluster based on resource demands. It integrates with Auto Scaling groups and requires minimal administrative overhead.
- ☐ Use an AWS Lambda function to resize the EKS cluster automatically.
Using a Lambda function for resizing adds complexity and administrative overhead compared to the Cluster Autoscaler.
- ☐ Use an Amazon EC2 Auto Scaling group to distribute the workload.
While an Auto Scaling group is fundamental to the Cluster Autoscaler, it does not independently solve the issue of Kubernetes-aware node scaling.

Overall Explanation:

The Kubernetes Cluster Autoscaler is the best solution. It monitors the Kubernetes scheduler for pending pods that can't be scheduled due to insufficient resources. The Autoscaler then automatically adjusts the size of the Auto Scaling group to accommodate the workload. This approach requires the least administrative overhead as it directly integrates with the Kubernetes cluster and automatically manages the scaling process. It handles the dynamic scaling needs of the EKS cluster without manual intervention. Alternatively consider Karpenter or EKS Managed Node Groups with autoscaling enabled.

However, Kubernetes Cluster Autoscaler is generally a better choice for least administrative overhead.

 [View Discussions](#)

 [Ask AI](#)

[← Back to Exam](#)



Marketing

[Affiliate Program](#)

[Unlimited Access](#)

[Business Unlimited Access](#)

Support

[Contact Us](#)

team@itcertifications.io

Company

[Udemy Alternative](#)

[Whizlabs Alternative](#)

[TutorialsDojo Alternative](#)

Legal

[Terms of service](#)

[Privacy policy](#)

[Refund policy](#)

[Shipping policy](#)



© 2026, ITcertifications Technologies. All rights reserved.